Development of Secure File Encryption Tools using AES256 to avoid Unauthorized Access

By

STEVEN WONG KHAI CHEE



FACULTY OF COMPUTING AND
INFORMATION TECHNOLOGY

TUNKU ABDUL RAHMAN UNIVERSITY OF
MANAGEMENT AND TECHNOLOGY
KUALA LUMPUR

ACADEMIC YEAR
2024/25

Development of Secure File Encryption Tools using AES256 to avoid Unauthorized Access

By

Steven Wong Khai Chee

Supervisor: Dr Muhammad Naim Bin Abdullah

A project report submitted to the
Faculty of Computing and Information Technology
in partial fulfillment of the requirement for the
Bachelor of Information Technology (Honours) in Information Security

Department of Information Systems And Security
Faculty of Computing and Information Technology
Tunku Abdul Rahman University of Management and Technology
Kuala Lumpur

Copyright by Tunku Abdul Rahman University of Management and Technology.
All rights reserved. No part of this project documentation may be reproduced, stored in retrieval system, or transmitted in any form or by any means without prior permission of Tunku Abdul Rahman University of Management and Technology.

Declaration

The project submitted herewith is a result of my own efforts in totality and in every aspect of the project works. All information that has been obtained from other sources has been fully acknowledged. I understand that any plagiarism, cheating or collusion or any sorts constitutes a breach of TAR University rules and regulations and would be subjected to disciplinary actions.



STEVEN WONG KHAI CHEE

Choose an item.

ID: 22WMR02069

Abstract

This project aims to develop a secure file encryption tool using the AES256 encryption standard to prevent unauthorized access to sensitive data. The tool is designed with a user-friendly interface and incorporates the zero-knowledge principle, ensuring that sensitive information, such as user passkeys, is never stored. A web-based environment is employed to facilitate testing, initially focused on TARUMT university students, with a goal of evaluating the system's effectiveness and usability.

The project follows the Agile development framework, which allows for flexibility and iterative improvements throughout the planning, analysis, design, implementation, and testing phases. The system design includes advanced cryptographic techniques, robust key management, and a client-side encryption model that guarantees data privacy. The implementation involves the use of the AES256 algorithm, paired with a secure hashing technique for key generation, all executed within the user's browser to maintain adherence to the zero-knowledge principle.

In addition to the technical development, the project is validated through comprehensive testing, including unit, integration, and user acceptance tests, ensuring functionality, security, and user experience across different devices and browsers. The absence of server-side data handling enhances privacy and security, making the tool a significant contribution to cybersecurity. Future research opportunities lie in expanding the tool's functionalities, such as supporting larger files and conducting more extensive vulnerability assessments.

This project contributes to the field of encryption technology by providing a practical, scalable, and secure platform for file encryption, offering both theoretical insights and practical solutions for secure data handling in the digital age.

Acknowledgement

I'm profoundly grateful to Dr Muhammad Naim Bin Abdullah, my FYP (Final Year Project) supervisor, for his invaluable guidance and support throughout every stage of my research journey. His insightful suggestions and constructive feedback were instrumental during both the planning and implementation phases of this project. Dr Naim’s willingness to generously dedicate his time and expertise to assist me is something I hold in high regard.

Furthermore, I would like to extend my heartfelt thanks to the publishers of online network resources. Their altruistic sharing of knowledge has been a beacon of support for learners like myself. The resources they provide, coupled with the diligent efforts of their managers, have significantly enriched our understanding and facilitated our academic pursuits.

Lastly, but certainly not least, I wish to express my sincere appreciation to all those who have contributed to and inspired our work on this FYP project. Your encouragement, guidance, and inspiration have played a pivotal role in shaping the outcome of this endeavor. Each of you has left an indelible mark on this journey, for which I am truly grateful.

Table of Contents

Declaration ii

Abstract ii

Acknowledgement ii

Table of Contents ii

1 Introduction 1

1.1 Background....2
1.1.1 Symmetric Encryption....3
1.1.2 AES Encryption Standard....3
1.1.3 Zero-Knowledge Encryption....4
1.2 Problem Statement....4
1.2.1 Credentials Compromised....4
1.2.2 Insecure Encryption Protocol....5
1.2.3 Incompatibility of Encryption tools....5
1.3 Objective....6
1.3.1 To develop an encryption platform for encryption & decryption of files using AES256 standard. 6
1.3.2 To employ zero-knowledge principle in encryption & decryption management....6
1.3.3 To deploy the developed encryption platform in the web-based system to assess its effectiveness in safeguarding sensitive data....6
1.4 Scope and Limitations....7
1.5 Significance of Project....7
1.6 Organization of Report....9
1.7 Chapter Summary & Evaluation....11

2 Literature Review 12

2.1 Review of Relevant Theories and Concepts....13
2.1.1 IoT security system with modified Zero-Knowledge Proof algorithm for authentication....13
2.1.2 Evaluation of AES Encryption....14
2.1.3 AES in Image File Security....14
2.1.4 The Need of Zero-Knowledge Proof(ZKP)....15
2.1.5 A Survey on Various Cryptanalytic Attacks on the AES Algorithm....17
2.2 Study of Similar Existing Systems....19
2.2.1 Review of Similar Existing Systems....19
2.2.2 Comparison of these systems....22
2.3 Identification of Gaps in the Literature....24
2.3.1 Lack of Comprehensive Integration of Zero-Knowledge Proof(ZKP) Protocol....24
2.3.2 Insufficient Exploration of File Encryption Using AES256....25
2.3.3 Limited Consideration of User Experience and Accessibility....25
2.3.4 Inadequate Exploration of Secure File Encryption Platforms....25
2.4 Summary of Key Findings....26
2.5 Chapter Summary & Evaluation....27

3 Methodology and Requirements Analysis 28

3.1 Project Methodology....29
3.1.1 Planning Phase....31
3.1.2 Analysis Phase....31
3.1.3 Design Phase....31

3.1.4 Implementation Phase 31
3.1.5 Testing Phase 32
3.1.6 Deployment Phase 32

3.2 Requirements Gathering 32
3.2.1 Techniques used for system requirements 32
3.2.2 Functional Requirements 33
3.2.3 Non-functional Requirements 33

3.3 Requirements Analysis 34
3.3.1 Analysis and prioritization of requirements 34
3.3.2 Identification of dependencies and conflicts requirements 35

3.4 Chapter Summary and Evaluation 36

4 System Design 37

4.1 Architecture Design 38
4.1.1 Use Case Diagram 39
4.1.2 Flowchart Diagram 40
4.1.3 Activity Diagram 42
4.1.4 Sequence Diagram 46
4.1.5 Deployment Diagram 49
4.1.6 Class Diagram 50

4.2 Database Design 52
4.2.1 Understanding Client-Side Encryption 53
4.2.2 Real-Scenario Handling of user's pass key 53

4.3 User Interface Design 54
4.3.1 Wireframe 54
4.3.2 Prototype Design 57

4.4 Chapter Summary and Evaluation 59

5 Implementation and Testing 61

5.1 Overview of System Implementation 61
5.1.1 System Usage 62
5.2 Front-End Implementation 68
5.2.1 User Interface(UI) code snippets 68
5.3 Core Implementation Components 70
5.3.1 Encryption Process Implementation 70
5.3.2 Decryption Process Implementation 73
5.3.3 Security Measures in Encryption and Decryption 75
5.4 Server and Application Implementation 76
5.4.1 Flask Web Application 76
5.4.2 Application Logic Flow 76
5.5 Testing Strategies 78
5.5.1 Unit Testing 78
5.5.2 Integration Testing 79
5.5.3 System Testing 80

5.5.4 User Acceptance Testing(UAT) 80
5.6 Chapter Summary and Evaluation..... 82

6 System Deployment 83

6 System Deployment(Not Applicable)..... 84

7 Discussions and Conclusion 852

7.1 Summary..... 86
7.2 Achievements..... 86
7.3 Contributions..... 86
7.4 Limitations and Future Improvements..... 87
7.5 Issues and Solutions..... 88

References 89

Appendices 91

Chapter 1

Introduction

Introduction

1.1 Background

Encryption has become a standard practice in today's digital world to keep various aspects of computer systems secure. Almost every technology we use involves data encryption, especially when transferring data, managing network traffic, and protecting information. So how does encryption work? Encryption works by converting data from a readable format into an unreadable encoded format (Kaspersky, 2020). When data is in this unreadable format, no one can understand its content, keeping it protected and hidden. The encrypted data can only be read or processed after it's been decrypted, usually using a personal passkey known only to the user who initiated the encryption process.

Encryption is essential in today's digital landscape for safeguarding sensitive information and ensuring secure communication channels. After all, encryption is a foundational element of cybersecurity (Poller, 2023). Encryption prevents unauthorized access and protects data privacy, including personal, financial, and proprietary information, thus preserving data confidentiality. It serves as a critical defense against cyber threats, reducing the risk of data breaches and identity theft. Moreover, encryption facilitates compliance with regulations and industry standards governing data protection, fostering trust among customers and stakeholders.

This project will be focusing on symmetric encryption as it is widely used in various aspects of our modern world to secure sensitive information and communications. For instance, in online banking and e-commerce transactions, symmetric encryption ensures that financial data, such as credit card numbers and personal details, is protected from unauthorized access during transmission over the internet. Similarly, in messaging applications and email services, symmetric encryption is employed to safeguard the privacy of communications between users, ensuring that messages remain confidential and cannot be intercepted by malicious actors.

Moreover, symmetric encryption plays a crucial role in securing data stored on electronic devices, such as computers, smartphones, and portable drives. By encrypting files and folders with symmetric encryption algorithms, individuals and organizations can prevent unauthorized access to their data in case their devices are lost, stolen, or compromised.

1.1.1 Symmetric Encryption

Symmetric encryption is a type of encryption key management solution where only one key (a secret key) is used to both encrypt and decrypt electronic data. ((Smirnoff & Turner, 2020). In this method, the sender and receiver share a single secret key, which is kept confidential between them. When the sender wants to encrypt a message or data, they use this secret key to transform the plaintext into ciphertext, which appears as gibberish to anyone who intercepts it without the key. Upon receiving the encrypted data, the receiver uses the same secret key to decrypt it back into its original plaintext form.

One of the main advantages of symmetric encryption is its efficiency and speed, making it suitable for encrypting large amounts of data. Additionally, because only one key is required for both encryption and decryption, symmetric encryption is relatively straightforward to implement and computationally efficient. Symmetric encryption provides a powerful and efficient means of securing data communication and storage, particularly when combined with secure key management practices. It is widely used in various applications, including secure communication over networks, data storage encryption, and protecting sensitive information in databases.

1.1.2 AES Encryption Standard

The AES Encryption algorithm (also known as the Rijndael algorithm) is a symmetric block cipher algorithm with a block/chunk size of 128 bits and It utilizes different key lengths (Jena, 2023), which can be either 128 bits, 192 bits, or 256 bits, depending on the particular variant chosen. The algorithm works by dividing the data into blocks and applying a series of substitution, permutation, and mixing operations to each block. These operations are performed in multiple rounds, with the number of rounds depending on the key size used (128-bit, 192-bit, or 256-bit). AES is known for its robustness, efficiency, and speed, making it suitable for a wide range of applications, including securing data in transit over networks, encrypting files on storage devices, and protecting sensitive information in databases. AES has become the de facto standard for encryption and is widely adopted by organizations and industries worldwide due to its strong security features and compatibility across different platforms and systems.

1.1.3 Zero-Knowledge Encryption

Zero-knowledge encryption has been around for a while, but its use has increased in the last few years, this is particularly true for consumer data storage services (Ware, 2022).

Zero-knowledge encryption keeps your data safe and private when using services like storing files online or managing passwords. Your data is scrambled using keys made from your main password and secret keys stored on your device. The app never keeps your data in plain, readable form, and the service's servers never get it that way either. This means only you can unscramble your data. Even if there's a security problem with the service, your data stays safe because no one else, not even the service provider, knows your password. This is why it's called zero-knowledge—only you know the secret (Ware, 2022). One of the way of how it is being applied on Cloud service is like when a user create a password on Cloud service, the password will be generated a hash value from their device to the Cloud Service to be saved, means that the Cloud service provider only knows the hash value of the password but not the original password written by the user, now only the user know the password.

1.2 Problem Statement

1.2.1 Credentials Compromised

Sensitive and private files and documents that are not encrypted remain exposed to several potential threats, including unauthorized access, data breaches, and identity theft.

Without using file encryption software, your files are much more at risk, complying with regulations will ultimately be more difficult, and security can be compromised across the board (Forta, 2022). Individuals and organizations risk compromising their confidential information, such as financial records, personal data, and proprietary documents if they continue to dive into the IT world without using encryption tools. Someone out there is trying to take advantage of our personal data—but who? The way we talk about privacy often disguises the true identity of the would-be data thief.(Bless, 2023).

Nowadays, people mostly use digital tools instead of paper to keep records. This means important stuff like bank info, finances, and health details end up on their computers, often in places like Sticky Notes or Notepad on Windows 10, thus are all easily being compromised if their electronic devices were hacked or stolen.

1.2.2 Insecure Encryption Protocol

Existing encryption tools or services widely use symmetric standards for encryption, which, while effective, may not provide complete security. In many cases, service providers store users' credentials, such as personal passkeys, which poses a significant risk of compromise. For instance, if a service provider's database is breached, attackers could potentially gain access to users' passkeys, undermining the security of encrypted data. Potential vulnerability in the current era is that with the attack on data breach is still ongoing, the data breach could expose millions of users' passkeys, leading to unauthorized access to encrypted files and compromising users' sensitive information. One of the great ways is to implement zero-knowledge encryption so no passkey will be stored on the server's side. However, many popular cloud storage services, such as Google Drive and Dropbox, don’t use zero-knowledge encryption and retain access to your files. (Wolford, 2023). Some companies don't apply the zero-knowledge principle because they want to make managing files easier and avoid wasting data space if clients forget their credentials for encrypted data, resulting in a less secure encryption protocol to protect the user's passkey.

1.2.3 Incompatibility of Encryption tools

As more people rely on electronic devices and transition to a paperless lifestyle, the need to secure sensitive information stored on phones or computers becomes increasingly critical. Individuals often save confidential data, such as banking passwords or health records, within their device's operating system, posing a significant risk if the device is compromised. Despite the availability of numerous encryption tools on the market, many users encounter challenges due to incompatibility of device’s setting in running encryption service, discouraging them from prioritizing encryption for security. For example, a file that's been encrypted from a smartphone is sometimes not supported and can’t be decrypted on a software application on a computer, and the other way around. A large portion of executives don't want their data encrypted because they view encryption as being too complicated to use. (Rebecca , 2020). For instance, popular encryption tools like VeraCrypt offer robust encryption services but are hindered by complex settings and overwhelming sub-functions, leading to usability issues and a reluctance to adopt encryption practices.

1.3 Objective

1.3.1 To develop an encryption platform for encryption & decryption of files using AES256 standard

The primary objective of this project is to design and implement an encryption platform capable of securely encrypting and decrypting files using the AES256 standard. This platform will utilize advanced cryptographic techniques to ensure the confidentiality and integrity of sensitive data. Specific features of the platform will include user-friendly interfaces for file selection, encryption, and decryption processes, as well as robust key management mechanisms to safeguard encryption keys. Additionally, the platform will support various file formats and sizes, enabling users to encrypt a wide range of file types securely..

1.3.2 To employ zero-knowledge principle in encryption and decryption management.

In addition to implementing AES256 encryption, this project aims to incorporate the zero-knowledge principle into the encryption and decryption processes. In the context of encryption, this principle will enhance security by ensuring that the encryption platform does not have access to users' plaintext data or encryption keys. Instead, encryption and decryption operations will be performed locally on the user's device, with the platform facilitating the exchange of encrypted data without compromising confidentiality. By leveraging zero-knowledge techniques, the encryption platform will provide an additional layer of protection against unauthorized access and data breaches, reinforcing the overall security posture of the system.

1.3.3 To deploy the developed encryption platform in the web-based system to assess its effectiveness in safeguarding sensitive data.

Once the encryption platform is developed, the next objective is to integrate it into a web-based system and evaluate its effectiveness in safeguarding sensitive data in real-world scenarios. The deployment phase will involve implementing the encryption platform within a secure web application environment, allowing users to upload, encrypt, and securely store files online. Through extensive testing and user feedback, the performance, usability, and security features of the encryption platform will be assessed to ensure its effectiveness in protecting sensitive data against various threats, including unauthorized access and data breaches. Additionally, measures will be taken to address any potential vulnerabilities or usability issues identified during the deployment phase, with the ultimate goal of delivering a reliable and user-friendly encryption solution for web-based applications.

1.4 Scope and Limitations

The usage of this developed project (Secure Encryption tool) will be limited to TARUMT university students as security testing of the efficiency of the tool. The type of encryption algorithm involved will be the AES256 standard. The encryption scope is limited to the encryption of document file and text type data only first, as my focus is to ensure the total efficiency of success and secure encryption of such data type.

This project is limited to the resources, expertise, and time available within the academic setting. The project aims to develop an encryption platform utilizing the AES256 standard and incorporating the zero-knowledge principle for encrypting and decrypting files. The scope includes designing and implementing the encryption platform, testing its functionality, and assessing its effectiveness in safeguarding sensitive data. However, due to constraints such as limited access to advanced technologies and professional expertise, the project may have certain limitations. These limitations may include constraints on the complexity of the encryption algorithms implemented, the sophistication of user interfaces, and the scalability of the solution. Additionally, time constraints and academic commitments may limit the depth and breadth of the research conducted and the extent of testing and validation performed. Despite these limitations, the project aims to deliver a functional and effective encryption platform within the confines of the academic environment, contributing to knowledge and skill development in the field of cybersecurity.

1.5 Significance of Project

This project holds significant importance in addressing pressing concerns surrounding data security and privacy in today's digital age. By developing an encryption platform utilizing the AES256 standard and incorporating the zero-knowledge principle, this project aims to enhance the protection of sensitive information and mitigate the risk of unauthorized access and data breaches. The importance of this project can be summarized as follows:

- Enhanced Data Security

  With the proliferation of cyber threats and data breaches, the need for robust encryption solutions has never been greater. This project seeks to bolster data security by providing

individuals and organizations with a reliable encryption platform capable of safeguarding sensitive data against unauthorized access and exploitation.

- Preservation of Privacy

  In an era marked by increasing concerns about privacy and data protection, the development of an encryption platform using the zero-knowledge principle underscores a commitment to preserving user privacy. By ensuring that only the user has access to their plaintext data and encryption keys, the platform empowers individuals to maintain control over their personal information and mitigate the risk of privacy violations.

- Empowering Users:

  By offering a user-friendly encryption platform, this project aims to empower individuals with the tools and knowledge needed to protect their data effectively. Through intuitive interfaces and streamlined processes, users can encrypt and decrypt files with ease, without the need for specialized technical expertise. This empowerment fosters a sense of autonomy and control over one's digital assets, promoting confidence in online activities and interactions.

- Compliance with Regulations:

  In an increasingly regulated environment, compliance with data protection laws and industry standards is paramount. By providing a robust encryption solution, this project enables organizations to meet regulatory requirements related to data security and privacy, thereby mitigating legal risks and potential liabilities associated with non-compliance.

- Trust and Confidence

  Ultimately, this project aims to instill trust and confidence in digital interactions and transactions by ensuring the confidentiality, integrity, and authenticity of sensitive information. By offering a reliable encryption platform, users can engage in online activities with greater peace of mind, knowing that their data is secure and protected against unauthorized access and exploitation.

1.6 Organization of Report

This report consisted of 7 chapters and the sequence is arranged as follows:

- Chapter 1: Introduction

  This chapter provides an overview of the project, including its background, objectives, significance, and scope. It sets the stage for the rest of the report by introducing the topic and outlining the key aspects of the project.

- Chapter 2: Literature Review

  In this chapter, a comprehensive review of relevant literature and existing research in the field of encryption, data security, and related technologies is presented. The literature review serves to contextualize the project within existing knowledge and identify gaps and areas for further investigation.

- Chapter 3: Methodology and Requirements Analysis

  This chapter outlines the methodology employed in the project, including the research approach, data collection methods, and analysis techniques. It also provides a detailed analysis of the project requirements, including functional and non-functional requirements, to guide the system design and implementation process.

- Chapter 4: System Design

  In this chapter, the design of the encryption platform is presented in detail. This includes the architectural design, database schema, user interface design, and other technical specifications. The system design chapter serves as a blueprint for the implementation phase of the project.

- Chapter 5: Implementation and Testing

  This chapter focuses on the implementation of the encryption platform and the testing procedures employed to ensure its functionality, reliability, and security. It includes details of the development process, software tools used, and the results of various testing methods employed.

- Chapter 6: System Deployment

  In this chapter, the process of deploying the encryption platform in a real-world environment is described. This includes considerations for deployment, configuration, and integration with existing systems or infrastructure. The system deployment chapter discusses the practical aspects of making the encryption platform operational.

- Chapter 7: Discussions and Conclusion

  The final chapter of the report presents a discussion of the project findings, including an analysis of the results, implications for theory and practice, and recommendations for future research. It concludes with a summary of the key findings and contributions of the project, along with reflections on the overall project experience.

1.7 Chapter Summary & Evaluation

In this opening chapter, this report sets the groundwork by highlighting the pivotal role of encryption in safeguarding sensitive information in today's digital landscape. It commences with an overview of encryption, elucidating its fundamental principles and significance in ensuring data security and privacy. Encryption is underscored as an essential component of cybersecurity, protecting data during transmission and storage, and enabling compliance with regulations. This chapter discusses the project's focus on symmetric encryption, particularly the AES256 standard, and introduces the concept of zero-knowledge encryption as an additional security measure. It outlines the problem statement, identifying risks associated with compromised credentials and insecure encryption protocols, as well as challenges related to the compatibility of encryption tools. The chapter concludes with an overview of the project's objectives, emphasizing the development of an encryption platform, integration of zero-knowledge principles, and deployment in a web-based system. Overall, this introductory chapter provides a comprehensive overview of the project's context, objectives, and significance in addressing critical issues in data security and privacy.

Chapter 2

Literature Review

2 Literature Review

The literature review serves as a critical component of any research endeavor, offering insights into existing knowledge, theories, and solutions relevant to the project at hand. This section provides a comprehensive overview of the existing literature related to secure file encryption tools using AES256. It elucidates the purpose and significance of the literature review in guiding the research process, informing the project's objectives, and identifying gaps or areas requiring further exploration. The structure and organization of the literature review section are outlined to provide clarity and coherence in presenting the review findings.

2.1 Review of Relevant Theories and Concepts (or Relevant Features and Functions))

In this subsection, key theories and concepts pertinent to the development of secure file encryption tools are discussed. Theories related to encryption techniques, cryptographic algorithms, and principles of data security lay the foundation for understanding the research approach. Concepts such as symmetric encryption, AES encryption standards, and zero-knowledge encryption principles are explored to inform the design and implementation of the encryption platform.

2.1.1 Zero-Knowledge Proof (ZKP) algorithm in IoT security system.

Based on the literature review of the journal article titled “IoT security system with modified Zero-Knowledge Proof algorithm for authentication”, the modified Zero Knowledge Proof algorithm contributes to authentication in data transmission in IoT security systems by improving the performance and security of the authentication process. The algorithm enhances the speed of the authentication process, with the proposed method showing better performance compared to previous methods. It reduces the authentication process to approximately 5 milliseconds from both the client side and server side, which is a significant improvement over the previous method that took between 1-15 seconds for authentication.

Additionally, the modified Zero Knowledge Proof algorithm increases the range of values and calculations, with the values being much larger calculations so breaking into the security will be much more difficult(Soewito & Marcellinus, 2020). This is crucial for IoT security systems, especially for continuous data transmission, as it provides a higher level of security. The algorithm also introduces a one-time token mechanism, where the token data transmission changes with each transmission, further enhancing security.

In summary, the modified Zero Knowledge Proof algorithm significantly improves the speed, security, and range of values for authentication in data transmission in IoT security systems, making it a valuable contribution to enhancing the overall security of IoT networks.

2.1.2 Evaluation of AES Encryption

,

Based on the study's report titled "AES Encryption: Study & Evaluation", the strength of AES is considered highly secure, with 128-AES, 256-AES, and 512-AES being virtually impenetrable. The possibilities available for both ciphers exceed the number of atoms in the universe, making brute force attacks not feasible. AES is significantly stronger than DES, with 256-AES being 340 billion-billion-billion-billion times harder to crack than 128-AES (Yehya & Joudi, 2020). In terms of speed, DES is faster than AES during encryption, likely due to the smaller block size of DES (56 bits) compared to AES (128 bits). However, during decryption, the time is slightly higher for AES compared to DES. When it comes to memory requirements, the AES algorithm uses less memory than the DES algorithm due to its characteristics and implementation requirements.

Advantages	Disadvantages
a. AES is a strong security protocol used in both hardware and software. b. It offers key sizes of 128, 192, and 256 bits, making it resistant to hacking. c. Widely used in applications such as wireless communication, financial transactions, e-business, and encrypted data storage. d. It is one of the most commonly used commercial and open source security solutions.	a. AES uses simple and easily understandable algebraic structures. b. All blocks are encrypted in the same way at all times. c. It can be difficult to implement with software, especially in counter mode, considering both performance and security.

Table 2.1: The advantages and disadvantages of AES encryption

2.1.3 AES in Image File Security

Based on the journal article titled "Implementation of image file security using the advanced encryption standard method", the document discusses the implementation of image file security using the advanced encryption standard method. The authors, from various universities in Malaysia and Indonesia, emphasize the importance of document security in the digital era and the use of cryptography to prevent unauthorized access to important information. The article was received in July 2022, revised in March 2023, and accepted in April 2023.

The advanced encryption standard (AES) method is used in image file security by performing encryption and decryption processes on the image files. This method utilizes a symmetric key with a key length of 128, 192, or 256 bits to encrypt and decrypt data in 128-bit blocks. The AES

method employs a block cipher system and can perform symmetric key encoding operations with different operating modes, such as electronic codebook (ECB) and cipher block chaining (CBC).

In the context of image file security, the AES method is applied to secure image files from unauthorized access or tampering. The encryption process involves generating elements of RGBA values from each pixel in the image file with PNG format(Ikhwan et al., 2023). The encrypted image file is then packaged with one of the operating modes, namely the electronic codebook operation mode or cipher block chaining. The choice of operating mode can impact the accuracy and security of the encryption process.

During the decryption process, the encrypted image file is decrypted using the AES method with the corresponding key and operating mode. The decrypted image is then displayed, and the original image file is reconstructed from the encrypted data.

Overall, the AES method is utilized to ensure the confidentiality and integrity of image files by applying strong encryption techniques and operating modes to protect the data from unauthorized access and manipulation.

2.1.4 The Need of Zero-Knowledge Proof (ZKP)

Based on the research article titled “Do You Need a Zero Knowledge Proof?” The research discusses the practical applications of Zero-Knowledge Proofs (ZKPs) in various scenarios, categorizes different types of ZKPs, and provides a flowchart methodology to determine the most suitable system for specific technical requirements. It also investigates major use cases of ZKPs, such as outsourcing computation, digital self-sovereign identity, and networking, and explores their broader implications and opportunities. The overall aim of the paper is to demystify the decision-making process involved in choosing the right ZKP.

Zero-Knowledge Proofs (ZKPs) have a wide range of major use cases with significant implications and opportunities. Below examples demonstrate the diverse applications of ZKPs and their potential to address privacy, security, and trust-related challenges in various domains.

a. Outsourcing Computation: ZKPs can be used to verify the correctness of computations outsourced to untrusted parties without revealing the sensitive data involved. This has implications for secure and verifiable cloud computing, where privacy and data integrity are crucial.
b. Digital Self-Sovereign Identity: ZKPs can enable the verification of identity-related information without disclosing the actual data, thus preserving privacy of the confidentiality of the underlying data(Ernstberger et al., 2024). This has implications for secure and privacy-preserving digital identity systems, where individuals can prove specific attributes about themselves without revealing unnecessary personal information.
c. ZKPs in Networking: ZKPs can be utilized in networking protocols to enable secure and private communication, authentication, and access control without revealing sensitive

information. This has implications for enhancing the privacy and security of communication protocols, access control mechanisms, and authentication processes.

The broader implications and opportunities of ZKPs extend to various domains, including:

a. Blockchain and Cryptocurrencies: ZKPs are used in blockchain applications for privacy-preserving transactions, efficient verification of state transitions, and secure interoperability between different blockchain networks.
b. Decentralized Applications (dApps): ZKPs are leveraged in dApps to enhance privacy, anonymity, and security in various functionalities such as decentralized finance (DeFi), on-chain gaming, and privacy-centric applications like smart contract mixers.
c. Proof-of-Storage: ZKPs can be applied in proof-of-storage schemes to enable clients to verify the integrity of stored data without revealing the actual content, thus ensuring data security and privacy.
d. Machine Learning and AI: ZKPs are being integrated with machine learning to address trustworthiness issues in outsourced learning, federated learning, and to facilitate verifiable machine learning model execution without exposing sensitive data.

The flowchart methodology mentioned in the document for determining the most suitable Zero-Knowledge Proof (ZKP) system involves a decision tree. This decision tree is designed to assist in the high-level decision-making process when choosing a proof system for an application.

The tree bifurcates into multiple decision paths based on the requirements of the application at hand, such as the necessity for a constant proof size or non-interactivity.

It bifurcates into multiple decision paths based on the requirements of the application at hand, such as the necessity for a constant proof size or non-interactivity(Ernstberger et al., 2024). The decision tree assumes that a computation is carried out between a prover and a verifier, and it provides guidance on selecting the appropriate ZKP system based on specific technical application requirements.

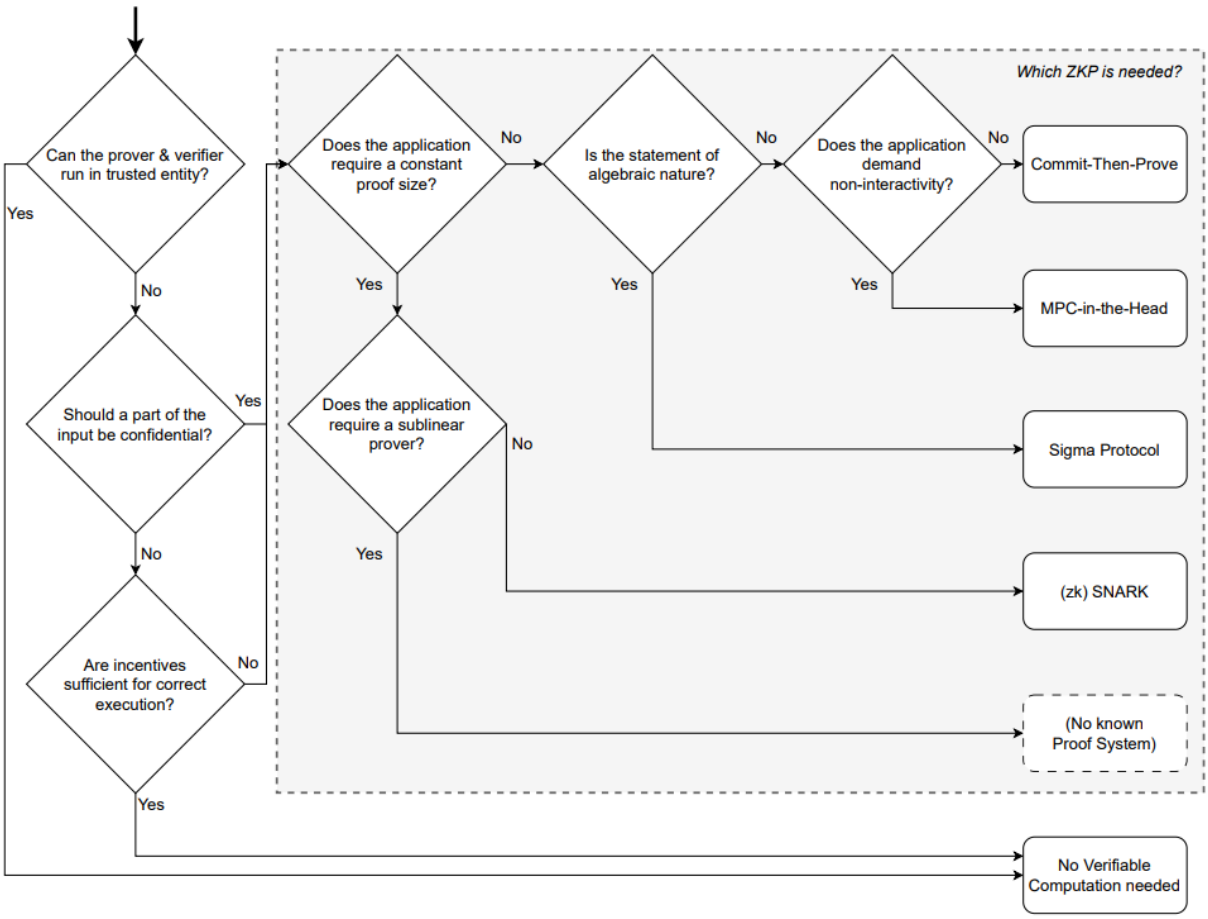


Figure 2.1: Decision tree to ease the high-level decision-making process (Ernstberger et al., 2024).

2.1.5 A Survey on Various Cryptanalytic Attacks on the AES Algorithm

The document discusses various cryptanalytic attacks on the Advanced Encryption Standard (AES) algorithm, which is widely used for securing classified information in military and banking services. The research examines theoretical attacks on the AES algorithm and compares them with the brute force attack for time and data complexity. While these attacks have not been able to completely break the AES algorithm, they are important for testing its security and obtaining the secret key. The paper provides a comprehensive survey of existing cryptanalytic attacks on the AES algorithm.

Theoretical attacks on the AES algorithm include various cryptanalytic techniques such as differential cryptanalysis, linear cryptanalysis, square attack, collision attack, and impossible differential attack. These attacks are aimed at breaking the AES algorithm to obtain the secret key or decrypt the encrypted data. For example, differential cryptanalysis focuses on analyzing the differences in plaintexts and the corresponding differences in ciphertexts to deduce information

about the key. Linear cryptanalysis exploits linear approximations in the encryption process to recover the key. The square attack, collision attack, and impossible differential attack are also methods used to exploit weaknesses in the AES algorithm.

These attacks are theoretical in nature and are used to test the security of the AES algorithm. While most of these attacks have been unable to completely break the AES algorithm, they are important for understanding its vulnerabilities and improving its security.

The outcomes of the attack on AES-256 are as follows:

Attack	**Key Size**	**No. of Rounds**	**Time Complexity**	**Data Complexity**	**Memory**	**References**
Square	All	6	2^{72}	2^{32}	2^{32}	Yu and Wei [2009]
Collision	256	7	2^{140}	2^{32}	2^{32}	Gilbert and Minier [2000]
Impossible Differential	128	6	2^{122}	$2^{91.5}$	2^{89}	Z'aba and Maarof [2006b]
Impossible Differential	192	7	2^{186}	2^{92}	2^{153}	Z'aba and Maarof [2006b]
Impossible Differential	256	7	$2^{250.5}$	$2^{92.5}$	2^{153}	Z'aba and Maarof [2006b]
Boomerang	128	5	2^{39}	2^{39}	2^{33}	Li et al. [2019]
Boomerang	128	6	2^{71}	2^{71}	2^{33}	Li et al. [2019]

Table 2.2: Summary of various attacks on AES Algorithm (Zodpe and Shaikh, 2021)

a. Impossible Differential Attack: The attack requires $2^{92.5}$ chosen plaintexts with a time complexity of $2^{250.5}$ .It is capable of breaking the AES-256 algorithm in seven rounds.
b. Boomerang Attack: This attack reduces the data complexity to 2^{71} in the sixth round, with a time complexity of 2^{71} for each reduced round and a memory complexity of 2^{33}.

According to Zodpe and Shaikh (2021), it is observed from the table that due to the high time and data complexity required for the cryptanalysis of AES algorithm, the algorithm is still secure. However, these attacks still demonstrate potential vulnerabilities in the AES-256 algorithm, highlighting the need for continued research and vigilance in the field of cryptography.

2.2 Study of Similar Existing Systems

2.2.1 Review of Similar Existing Systems

A. Cryptii

Cryptii is a dynamic online tool licensed under MIT, developed by Fränz Friederes, aimed at simplifying the complexities of code decoding and encoding. With its open-source nature, Cryptii has evolved into a versatile platform offering a myriad of functionalities tailored to meet the diverse needs of users exploring encryption. This is official link to access to the Cryptii web services: https://cryptii.com/

The user experience within Cryptii is characterized by its intuitive interface, fostering an environment of exploration and experimentation. Whether users are novices or seasoned professionals, Cryptii's simplicity invites engagement, making coding and decoding activities not only efficient but also enjoyable. Moreover, the transition to a new interface underscores Cryptii's commitment to user privacy and security, ensuring that all conversions occur directly within the user's browser.

Cryptii's feature set encompasses a wide range of capabilities, from basic encoding to more advanced hashing techniques. While its focus on historic encryption methods may not align with modern cryptographic standards, Cryptii remains dedicated to incorporating cutting-edge techniques, keeping pace with the evolving landscape of encryption technology. Despite some limitations, such as being unable to encrypt file-size documents, Cryptii's core functionalities provide a solid foundation for users to explore the intricacies of encryption.

In terms of safety and security, Cryptii emphasizes the importance of browser integrity, urging users to exercise caution, particularly with third-party extensions（Baldwin, 2023）. However, with

no reported incidents of malware, Cryptii instills confidence in its safety measures, further supported by its collaborative approach to customer support through GitHub and direct contact with the developer.

While Cryptii may not cater to the demands of modern cryptographers, its educational value and utility for developers are undeniable. As an accessible gateway to the world of encryption, Cryptii serves as a valuable resource for individuals seeking to understand the significance of encryption in today's digital landscape.

Interface	**Accessibility**	**Compatibility**	**Security Level**	**Customer support**
Easy to Use	Clone or download Or use the live version over at https://cryptii.com	Desktop version/ On and offline The web app is responsive and fully accessible on mobile devices, too.	Not secure	No available

Figure 2.3: Cryptii encryption service characteristics (Baldwin, 2023)

B. FreeCrypt (https://www.bat-soft.com/freecrypt-faq/)

FreeCrypt, offered by Bat-Soft, is a robust encryption solution designed to safeguard sensitive data while ensuring user accessibility. This is the official authorized link to access to the FreeCrypt web encryption service: https://freecrypt.org/ .The service prioritizes security and privacy, catering to individuals and organizations seeking reliable encryption methods.

a. Password Recovery Policy:

FreeCrypt adopts a stringent approach to password recovery, acknowledging the inherent complexity of encryption schemes. While recognizing the possibility of cracking encryption, FreeCrypt underscores the importance of data security over accessibility.

b. Cost and Registration:

True to its name, FreeCrypt is available at no cost for non-commercial use. However, registration options are available for those requiring additional technical support beyond

the provided FAQ. Commercial entities are required to contact Bat-Soft for licensing information.

c. File Size Limitations:

FreeCrypt currently supports the uploadable file size of maximum 1MB, operated in a web-based environment. Extensive testing validates its ability to process large files seamlessly, ensuring efficient encryption and decryption processes.

d. Security Measures:

FreeCrypt assures users of robust security measures, with passwords being case-sensitive to enhance security. While specific details of the encryption algorithm remain undisclosed(Baldwin, 2023), FreeCrypt's unique approach aims to deter potential hackers, prioritizing user data protection.

e. Compatibility and Reliability:

FreeCrypt's compatibility spans on a web-based platform, where anyone with a browser-supported app or software can access it, ensuring seamless integration and operation across various platforms. Extensive testing and user feedback affirm its reliability and performance in diverse environments.

C. File Encryptor | Decryptor by webbrowsertools.com

(https://webbrowsertools.com/file-encryptor/)

File Encryptor | Decryptor by webbrowsertools.com is a convenient and dependable tool designed to encrypt or decrypt files securely. Utilizing the native JavaScript web crypto API, this web application ensures that all encryption and decryption processes occur locally on the user's machine, enhancing privacy and security. File Encryptor | Decryptor by webbrowsertools.com stands out as a reliable and user-friendly encryption service. Its emphasis on privacy, security, and ease of use makes it a valuable tool for individuals seeking to protect their sensitive files.

a. Privacy and Security:

One of the standout features of this encryption service is its commitment to user privacy and security. By leveraging the native JavaScript web crypto API and refraining from using third-party libraries, the application eliminates the need for data transmission to external servers or cloud storage. This local processing approach enhances confidentiality and minimizes potential security vulnerabilities.

b. Ease of Use:

The user interface of File Encryptor | Decryptor is intuitive and user-friendly, making it accessible to individuals with varying levels of technical expertise. Users can simply drag and drop their files into the input area or utilize the file I/O popup window to select files for encryption or decryption. Additionally, the process of entering and confirming passwords is straightforward, facilitating smooth operation.

c. Compatibility and Accessibility:

This web application is compatible with all modern browsers, including Chrome, Firefox, Opera, and Edge, ensuring accessibility across a wide range of platforms. Furthermore, the availability of a browser extension enhances usability, allowing users to utilize the encryption service offline without requiring an internet connection.

2.2.2 Comparison of these systems

Web-app Encryption tool	**Cryptii**	**FreeCrypt**	**File Encryptor | Decryptor**	**StevHash**
Algorithm Used	Wide-variety	Disclosed	Disclosed	AES256 PBKDF2
Ease-of-Use	Nope	Yes	Yes	Yes
Compatibility	Web-based application	Web-based application	Web-based application	Web-based application
Supported file type	Text and Binary only	File type up to 1MB size only	File type only	File type, Photo, Music, Pdf, Document
Client-side encryption	Yes	Disclosed	Yes	Yes
PassKey Generation	Not required	Auto-Generated	User-input	User-Input
Zero-knowledge Protocol	No	No	Not Specified	Yes

Table 2.4: Comparison of each encryption platform service.

From the table above, several differences are evident among these similar systems. As generally displayed, all encryption tools here are web-based, meaning they can be accessed on smartphones or any computer device as long as it has a browser application. Almost each proposed tool provides ease-of-use in the standard encryption process, but not Cryptii, as it offers a lot of customizable settings for users to configure their specified encryption standard and algorithm. This complexity makes Cryptii less user-friendly compared to the others. FreeCrypt and File Encryptor | Decryptor both do not explicitly mention their algorithms used in the encryption and decryption process. Still, it can be considered a good security practice to prevent attackers from conducting specific algorithm-based attacks on the encrypted files.

In terms of supported services, both FreeCrypt and File Encryptor | Decryptor allows file encryption, but Cryptii only supports encryption of text and binary values. FreeCrypt has a limited file size of a maximum of 1MB currently for encrypting a file. For client-side encryption, both Cryptii and File Encryptor | Decryptor explicitly mentions in their service introductions that they apply client-side encryption services where there is no internal involvement of the server, especially regarding database usage. However, FreeCrypt does not explicitly mention whether it uses client-side or server-side encryption; the information about it is undisclosed.

All these three encryption tools also do not explicitly mention that they apply the Zero-knowledge protocol (ZKP) or principle in their encryption process. Even though Cryptii and File Encryptor | Decryptor performs client-side encryption only; it still doesn't fully qualify as fulfilling ZKP. For Cryptii, the encryption service doesn't require the user to input any passkey despite offering a wide variety of encryption algorithms. Notably, when users switch to applying the AES256 algorithm in Cryptii, users just have to insert text, and the output of the encrypted text is shown right away. Even though the produced output is not saved by Cryptii at all, users who copy the output encrypted text might find it unsafe as it does not incorporate a passkey, meaning that the copied encrypted text might be easily broken by an expert hacker.

For FreeCrypt, the encryption service automatically generates a hashed value for the user after the uploaded file has been encrypted. The user needs to save it for future decryption purposes; thus, this produced hashed value can be treated as the passkey. From this process, we might conclude that FreeCrypt is involved in server-side functionality as it generates and stores a known hash

value that acts as a passkey for each user who has encrypted their file on the platform. For this reason, FreeCrypt does not comply with ZKP as it is in control of passkey generation. Therefore, there's a risk that the service provider could potentially have access to or knowledge of the passkey, especially if the generation process is not transparent or if the passkey is transmitted to the user over an insecure channel. FreeCrypt, as the service provider, could potentially access or retrieve the passkey, undermining the zero-knowledge principle.

File Encryptor | Decryptor generates passkeys based on user input for passkey generation. This user-defined passkey is more secure as the service provider itself does not store the passkey information, but it is kept by the user themselves. File Encryptor | Decryptor also sets a condition requiring the user to insert a complex passkey so that the encryption process may proceed, thus increasing the complexity of the hashing algorithm, making the encrypted file more secure and unbreakable.

2.3 Identification of Gaps in the Literature

The analysis of the existing literature reveals several gaps, contradictions, and areas necessitating further exploration in the realm of secure file encryption tools utilizing AES256. These gaps and areas of interest serve as crucial focal points for the advancement of research in this domain. Below are the identified gaps along with explanations of how the project aims to address them:

2.3.1 Lack of Comprehensive Integration of Zero-Knowledge Proof(ZKP) Protocol

One noticeable gap in the literature is the absence of a comprehensive integration of the Zero-Knowledge Proof (ZKP) protocol within existing secure file encryption tools. While some studies briefly mention ZKP as a potential enhancement for authentication and security, there is limited exploration of its practical implementation in encryption platforms. The project aims to address this gap by incorporating a modified ZKP algorithm into the encryption process, thereby enhancing authentication mechanisms and bolstering the overall security of the file encryption tool.

2.3.2 Insufficient Exploration of File Encryption Using AES256

A notable gap in the existing literature is the lack of detailed investigation into the application of the AES256 standard specifically for encrypting image files. While AES256 is recognized as a strong encryption standard, there is limited research exploring its effectiveness and optimization specifically for image data encryption. The project aims to address this gap by focusing on the application of AES256 for encrypting image files. This involves examining how AES256 performs in encrypting various types of image files, assessing its strengths and weaknesses in this context, and exploring potential optimizations or specialized techniques tailored to image encryption while solely relying on AES256

2.3.3 Limited Consideration of User Experience and Accessibility

The literature review reveals a gap in the limited consideration of user experience and accessibility in existing secure file encryption tools. While the focus is primarily on the technical aspects of encryption algorithms and security protocols, there is less emphasis on user-friendly interfaces and intuitive design elements. The project aims to bridge this gap by prioritizing user experience in the development of the encryption tool, implementing intuitive interfaces, and providing clear documentation to facilitate ease of use for a wide range of users.

2.3.4 Inadequate Exploration of Secure File Encryption Platforms

Lastly, there is a gap in the literature concerning the absence of a comprehensive comparative analysis of various encryption platforms, including their strengths, weaknesses, and suitability for different use cases. While individual studies evaluate specific encryption tools or algorithms, there is a lack of holistic comparisons that consider multiple factors such as security, performance, usability, and scalability. The project intends to address this gap by conducting rigorous comparative evaluations of existing encryption platforms, identifying their respective strengths and limitations, and proposing recommendations for selecting the most suitable tool based on specific requirements and constraints.

2.4 Summary of Key Findings

This section synthesizes the main findings from the literature review and discusses how these findings inform the project's research objectives and approach. Synthesis of Main Findings:

A. AES Encryption Strengths and Weaknesses

The literature highlights the strengths of AES encryption, including its strong security protocols and resistance to hacking. However, it also identifies weaknesses such as its vulnerability to certain cryptanalytic attacks and challenges in implementation, particularly in counter mode.

B. Importance of Zero-Knowledge Proof (ZKP) Protocol

The review reveals the potential of ZKP in enhancing authentication and security in encryption platforms. Despite brief mentions in some studies, there is a lack of comprehensive integration of ZKP into existing tools.

C. Secure File Encryption Using AES256

While AES256 is recognized as a robust encryption standard, there is limited research on its application specifically for encrypting image files. Existing literature primarily focuses on general file encryption methods rather than exploring the unique challenges of image file encryption.

D. User Experience and Accessibility

The literature review identifies a gap in the consideration of user experience and accessibility in existing encryption tools. While technical aspects are emphasized, there is less emphasis on intuitive interfaces and clear documentation for users.

E. Comparative Analysis of Encryption Platforms

Existing research lacks comprehensive comparative analysis of encryption platforms, making it challenging for users to select the most suitable tool based on their requirements.

The findings from the literature review inform the project's research objectives and approach in several ways for what shall be included in my project development:

A. Integration of ZKP Protocol: The project will focus on integrating a modified ZKP algorithm into the encryption process to enhance authentication mechanisms and bolster overall security.
B. Implementation of AES256 for File Encryption: Building on the strengths of AES256, the project will explore its effectiveness and optimization for encrypting text, document files with better sizing, addressing the gap identified in the literature.
C. Supported text and document file data type for encryption: The project will be built based on its capability to support the encryption process of text word as well as document of varying size.
D. Comparative Evaluation of Encryption Platforms: The project will conduct rigorous comparative evaluations of existing encryption platforms, providing users with valuable insights into their strengths, limitations, and suitability for different use cases.

2.5 Chapter Summary & Evaluation

Chapter 2 provides a comprehensive overview of the existing literature on secure file encryption tools using AES256. The literature review highlights the strengths and weaknesses of AES encryption, explores the potential of Zero-Knowledge Proof (ZKP) protocol, and identifies gaps in the application of AES256 for image encryption. Additionally, the review emphasizes the importance of user experience and accessibility in encryption tools and calls for a comparative evaluation of existing encryption platforms. These findings inform the project's research objectives, which include integrating ZKP protocol, exploring AES256 for image encryption, prioritizing user experience, and conducting comparative evaluations. Overall, Chapter 2 lays the foundation for the project's research by synthesizing key findings from the literature review and shaping the direction of future investigations in secure file encryption.

Chapter 3

Methodology and Requirements Analysis

3 Methodology and Requirements Analysis

This chapter delves into the methodology and requirements analysis undertaken for developing the secure file encryption tool. It explores the chosen Agile methodology and its benefits, detailing each phase of the project lifecycle, including planning, analysis, design, implementation, testing, and deployment. The chapter also discusses techniques used to gather system requirements, classification of functional and non-functional requirements, and the process of requirements analysis, including evaluation, prioritization, and identification of dependencies and conflicts among the defined requirements. This comprehensive overview sheds light on the strategic approach adopted to develop a robust and user-friendly secure file encryption tool..

3.1 Project Methodology

For the development of the secure file encryption tool using AES256, the chosen methodology is Agile due to its adaptability and emphasis on continuous feedback and iterative development. This methodology is particularly suitable for projects where requirements may evolve or become clearer as development progresses, which is common in encryption and security-related projects.

The choice of Agile methodology was driven by the project's need for flexibility and frequent reassessment, which are crucial given the evolving nature of security threats and encryption technologies. Agile allows for adaptive planning, evolutionary development, early delivery, and continuous improvement, all of which enable rapid and flexible responses to change, which is essential for a project developing cutting-edge security solutions. Additionally, the iterative approach of Agile facilitates early detection of issues and integration of stakeholder feedback, thereby improving the final product's quality and relevance



Figure 3.1 : Agile Methodology Diagram

Agile methodology proved instrumental in several ways. Firstly, it accommodates the iterative nature of encryption tool development, where requirements may evolve or become clearer as the project progresses. For instance, as new encryption standards or security vulnerabilities emerge, Agile provides the framework to swiftly incorporate these changes into the project roadmap. Moreover, Agile's emphasis on early and frequent deliveries aligns well with the need to validate and refine security features iteratively. Instead of waiting for a lengthy development cycle to complete before obtaining feedback, Agile enables stakeholders to interact with functional prototypes early on, facilitating early detection of issues and integration of stakeholder feedback. Consider a scenario where during the initial phases of development, user testing reveals a usability issue with the encryption tool's interface. With Agile, this feedback can be promptly addressed in subsequent iterations, ensuring that the final product meets not only security standards but also user experience expectations. Furthermore, the continuous improvement aspect of Agile ensures that the encryption tool remains relevant and effective in mitigating emerging security threats. As new encryption techniques or security best practices emerge, Agile allows for the seamless integration of these advancements into the project's development process. In essence, Agile methodology serves as an enabler for the successful development of the secure file

encryption tool by providing the flexibility, adaptability, and responsiveness required to navigate the complexities of modern security challenges.

3.1.1 Planning Phase

In the planning phase, the project scope was defined, including the key deliverables and the timeline. The project was broken down into 2 phases, each lasting 4 months, allowing regular reassessment and adjustment of the project plan based on testing results. Key activities included defining the objectives, problems that arise regarding secure encryption tools, making comparisons, selecting technologies (such as AES256 and Zero-Knowledge Proof for encryption), and planning the overall architecture of the encryption platform.

3.1.2 Analysis Phase

This phase involved a deep dive into the existing literature on AES256 encryption and similar systems, as outlined in Chapter 2. The analysis helped identify gaps in current technologies, such as the need for better integration of user-friendly interfaces and robust security measures. This phase provided a theoretical foundation and validated the need for a new encryption tool tailored to modern security demands.

3.1.3 Design Phase

The design phase focused on creating detailed architecture for the encryption tool. This included the development of system architecture diagrams, user interface designs, and the integration approach for Zero-Knowledge Proof technology. The aim was to design a system that is both secure and easy to use, addressing both the technical and user experience aspects identified in the literature review. The design phase also included the design of activity diagram, use case diagram and sequence diagram to ensure the smooth and logical flow of the process in using the encryption tool.

3.1.4 Implementation Phase

During implementation, coding practices were aligned with industry standards to ensure security and maintainability. The AES256 encryption algorithm was integrated into the platform, with special attention to ensuring that the system could handle the encryption of files with different sizes efficiently. Regular code reviews and pair programming sessions were conducted to maintain code quality and incorporate new insights as development progressed.

3.1.5 Testing Phase

The testing phase tends to validate the functionality, security and user experience of the encryption tool. Below are the standard testing of the application that will be conducted:

a. Unit Testing: Individual components were tested for expected functionality.
b. Integration Testing: Different system modules were tested together to ensure they worked harmoniously.
c. System Testing: The entire system was tested to verify that it met the requirements specified in the planning phase.
d. User Acceptance Testing (UAT): End-users tested the system to ensure it met their needs and was user-friendly.

3.1.6 Deployment Phase

The deployment phase involved setting up the encryption tool in a web application environment. The initial deployment was closely monitored to address any immediate issues. The tool was initially deployed within a limited university scope, targeting students for initial usage. This phased approach allowed for iterative feedback and refinements before a broader rollout.

3.2 Requirements Gathering

Requirement gathering refers to the process of collecting, analyzing, and defining the needs and expectations of users for a particular project or system. In the context of the secure file encryption tool, requirement gathering involved identifying the essential functionalities and features necessary to meet good security standards.

3.2.1 Techniques used for system requirements

The system requirements for the secure file encryption tool were gathered through a comprehensive analysis of existing industrial standards and best practices through online sources. This involved examining established encryption protocols, such as AES256, and identifying key functionalities and features essential for ensuring robust security and usability. Additionally, insights from the literature review were leveraged to understand user needs, industry trends, and

potential areas for improvement in existing encryption solutions. By employing this technique, the project team could align the system requirements with industry benchmarks and user expectations, ensuring that the resulting encryption tool would meet both security and usability criteria.

3.2.2 Functional Requirements

Functional requirements for the secure file encryption tool encompassed a range of essential features and capabilities necessary for secure data encryption and decryption. These functional requirements included:

a. AES256 encryption and decryption functionality to ensure robust data security.
b. Support for various file formats, including text, binary, and image files, to accommodate diverse user needs.
c. Integration of Zero-Knowledge Proof (ZKP) protocol for enhanced authentication and privacy protection.
d. Compatibility with web-based platforms for seamless deployment and accessibility across different devices.
e. Error handling mechanisms to address potential issues and ensure smooth operation under different scenarios.

By defining these functional requirements, the project team established clear guidelines for the development of the encryption tool, ensuring that it would meet the core functionality expected by users while maintaining stringent security standards.

3.2.3 Non-functional Requirements

Non-functional requirements for the secure file encryption tool focused on aspects related to performance, security, and usability that are critical for ensuring the overall effectiveness and reliability of the system. These non-functional requirements included:

- Security requirements

  Ensuring compliance with industry-standard encryption protocols and best practices to safeguard sensitive data from unauthorized access or tampering. This encompassed

measures such as encryption key management, input validation, secure data transmission protocols, and protection against common security threats according to the latest OWASP Top 10 vulnerabilities threat.

- Performance requirements: Defining benchmarks for system performance, such as encryption and decryption speed, resource utilization, and scalability, to ensure optimal functionality under varying workloads and usage scenarios.
- Usability requirements: Incorporating user-centric design principles to enhance the user experience and accessibility of the encryption tool. This involved considerations such as intuitive interface design, clear error messaging, and support for multiple languages or accessibility features.

3.3 Requirements Analysis

Requirement analysis refers to the process of examining and evaluating the gathered requirements to ensure their feasibility, completeness, and consistency. This phase aims to refine and prioritize the requirements based on their importance to security, usability, and overall project success

3.3.1 Analysis and prioritization of requirements

Each requirement gathered during the requirements gathering phase is meticulously evaluated based on its criticality to security and usability.

- The requirement for AES256 encryption functionality is deemed crucial for ensuring robust data security, given its widely recognized strength in cryptographic applications.
- The integration of Zero-Knowledge Proof (ZKP) protocol is prioritized due to its significance in enhancing authentication and total privacy protection, addressing key security concerns identified in the project scope. ZKP safeguards user privacy and confidentiality, fostering trust in the encryption tool's capabilities.
- Support for various file formats are pivotal for ensuring the tool's versatility and practicality. Users commonly deal with diverse file types, including text documents, images, and binaries. Thus, the ability to encrypt and decrypt a wide range of formats ensures the tool's applicability across different scenarios, enhancing its utility and user satisfaction.
- Integration of error handling mechanisms is essential to maintain system integrity and user trust. In the event of unexpected errors or disruptions during encryption or

decryption processes, robust error handling mechanisms provide clear feedback to users, guiding them on how to resolve issues and ensuring uninterrupted operation.

- Compatibility with web-based platforms is vital for accessibility and convenience. With the increasing reliance on web technologies, enabling users to access the encryption tool via web browsers enhances its reach and usability. It allows users to utilize the tool across various devices without the need for additional installations or configurations, streamlining their workflow and promoting adoption.

Therefore, prioritizing these functional requirements is critical to delivering a secure file encryption tool that is versatile, and reliable.

3.3.2 Identification of dependencies and conflicts among requirements

Dependencies and conflicts among requirements are identified to anticipate potential challenges and ensure seamless integration of system functionalities. For example, the integration of the ZKP protocol for authentication may introduce dependencies with certain system functionalities, requiring adjustments to accommodate the specific authentication mechanism. Moreover, the support for various file formats is highly dependable on the effectiveness of implementing AES256 in the encryption process. The robustness of AES256 encryption relies on its ability to handle different types of files effectively. Therefore, the seamless integration of file format support is essential for ensuring the overall efficacy of the encryption tool in safeguarding data across diverse file types.

In the project development, potential conflicts may arise between performance requirements and resource utilization, where optimizing encryption and decryption speed may impact the system's scalability or compatibility with web-based platforms. Another potential conflict is that ZKP may pose usability challenges in certain scenarios. For instance, in the event of a user forgetting their key phrase for the encryption process, the secure file encryption tool would be unable to recover it. This limitation stems from the tool's adherence to ZKP standards, which mandate that user credentials are not stored within the system. Instead, only the hashed values of credentials are retained to fulfill ZKP requirements. As a result, while ZKP enhances security by minimizing data exposure, it also imposes restrictions on password recovery mechanisms, potentially impacting user experience and usability.

Furthermore, technique limitations, such as the constraint to utilize only freely accessible resources for development, may introduce conflicts with desired functionalities or security measures, necessitating careful consideration and adaptation of the project development.

3.4 Chapter Summary and Evaluation

Chapter 3 provides exploration of the methodology and requirements analysis employed in the development of the secure file encryption tool. Beginning with the rationale behind selecting Agile methodology, the chapter highlights its adaptability and iterative nature, crucial for addressing evolving security threats. Each phase of the project lifecycle, from planning and analysis to deployment, is meticulously outlined, emphasizing the importance of aligning project goals with industry standards and user expectations. The chapter also delves into the techniques utilized for gathering system requirements, distinguishing between functional and non-functional requirements essential for the tool's functionality, security, and usability. Moreover, the process of requirements analysis is thoroughly examined, emphasizing the evaluation, prioritization, and identification of dependencies and conflicts among the defined requirements. By prioritizing security, usability, and project success, Chapter 3 underscores the strategic approach adopted to develop a versatile and reliable secure file encryption tool.

Chapter 4

System Design

4 System Design

System design in web application development serves the crucial purpose of architecting the entire system to ensure it meets the functional requirements while also being scalable, reliable, and maintainable. In this chapter, the system design involves designing the structure, components, and interactions of the system, including databases, servers, and client interfaces. Diagrams included in this chapter are flowchart, activity diagram, user case diagram, sequence diagram, deployment diagram, class diagram, database design, wireframe and prototype interface. Ultimately, system design ensures that the web application functions smoothly, efficiently, and can adapt to future changes or growth.

Do take note that in this chapter, for our secure file encryption tool web application to be addressed more conveniently, we will be using the name StevHash as the name of our web application for now on.

4.1 Architecture Design

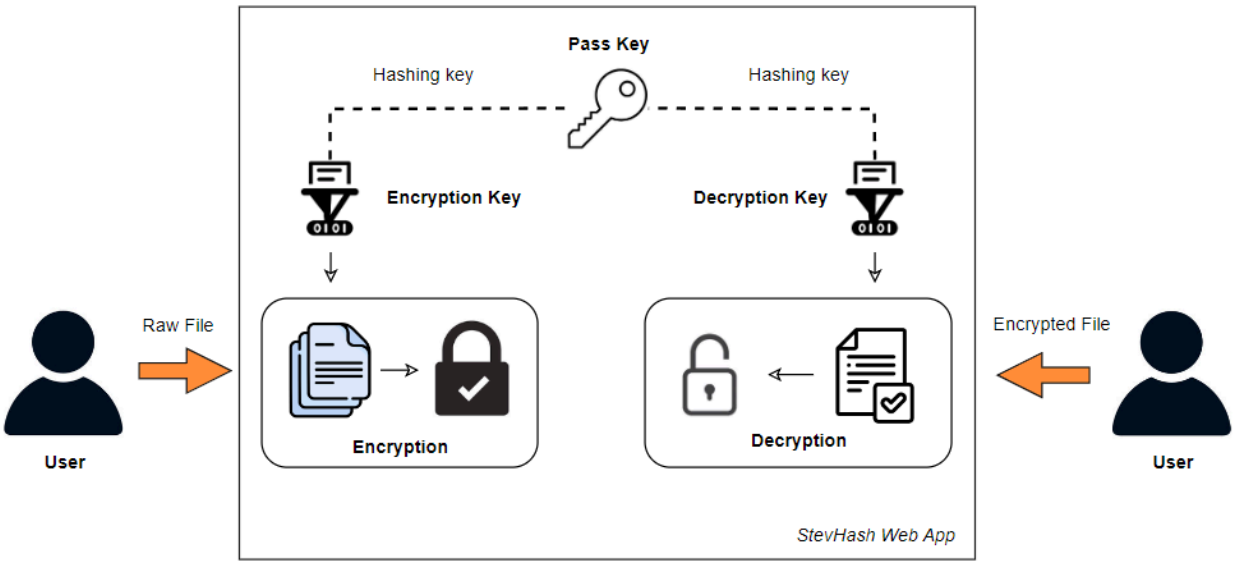


Figure 4.0: Concept flow of the StevHash web application.

The figure 4.0 displayed the basic concept of architecture design of the secure file encryption tool. In the figure, the same pass key is inserted by the user for both the encryption and decryption process. That particular pass key is only known by the user himself but not the application or any third party. The web application will generate a hash value from the pass key to be used as the encryption key or decryption

key for the input file. Users can input raw files for the encryption process or input encrypted files for the decryption process.

4.1.1 Use Case Diagram

The use case diagram is designed to illustrate the interactions between users (actors) and the system to show the different ways that users can use the system functionalities.

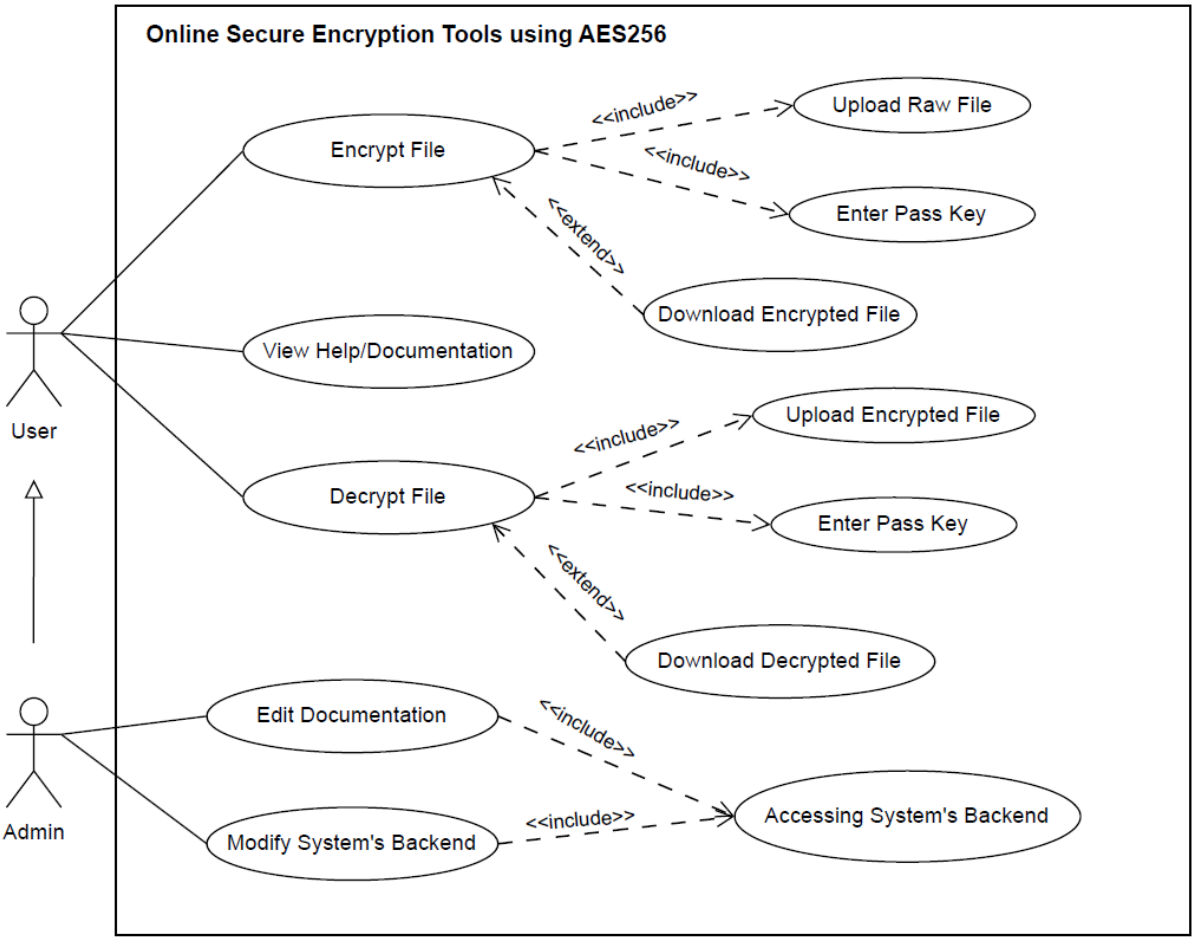


Figure 4.1: Use case diagram of the StevHash encryption web app service.

From the above use case diagram, there are mainly 2 actors in the usage of this StevHash web app which are the users and the admin. The user can perform actions which are to encrypt, decrypt files and view documentation of the web app. The documentation tends to outline the basic details for users to understand the operation and protocols used in this web app service. For the process of encrypting/decrypting a file, the user must first upload a file and insert a pass key. The generated output file is optional for the user to download or just ignore since the web application won't store the file. On the admin side, the admin can do whatever the user can do. But

additionally, the admin can generally perform an action which is accessing the web app system's backend and then edit documentation or the system structure of the backend such as modify source code.

4.1.2 Flowchart Diagram

The flowchart diagram represents the step-by-step logical flow of a process, highlighting decision points and the sequence of actions in a procedure.

A. Flowchart for Encryption Process

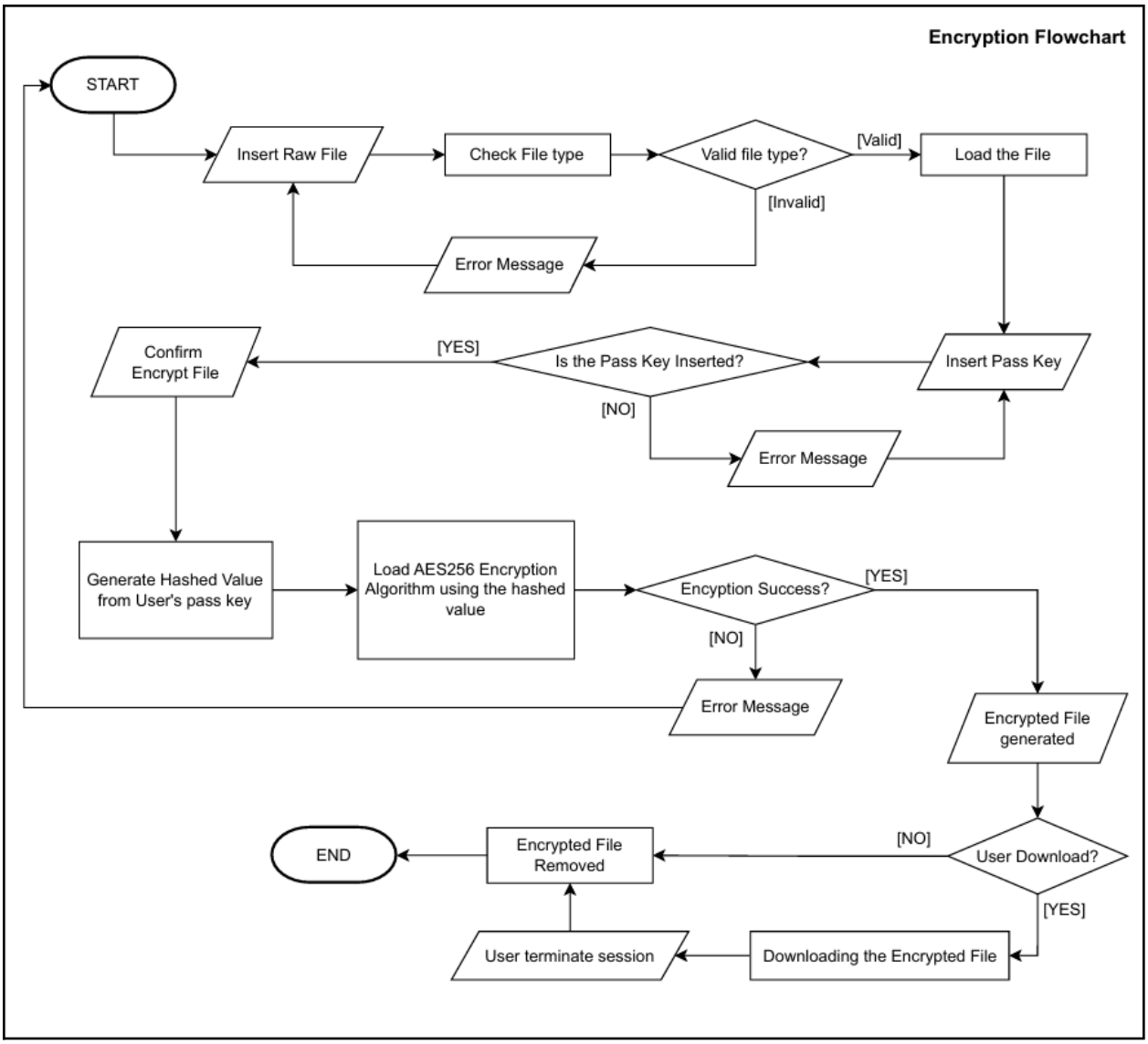


Figure 4.2: Flowchart diagram for encryption process.

For the encryption process as shown in the flowchart diagram, users are first required to insert a raw file. Then the system will check the validity of the file type whether it can be supported or not. If not, an error message will be displayed and prompt the user to re-insert valid files. If the inserted file type is valid, then the system will load the file. Users have to continue inserting the pass key. The system will check the validity of the passkey. With the valid pass key, the user can click to confirm the encryption execution. The system will first generate a hash value from the pass key to use as the encryption key. The encryption key is then used to load the AES256 algorithm to encrypt the raw file. Successful encryption process will generate an encrypted file to the user, and the user can choose to download it. In the event of the user ignoring the result and closing their web browser/ or terminating their web session in any way, the encrypted file will be removed right away.

Do take note that the pass key inserted by the user is to generate hash value, and that hash value is then being used directly as the encryption key for AES256. Secure hashing will be done with salted hash (added unique character to user's passkey first, then hash it with PBKDF2)

B. Flowchart for Decryption Process

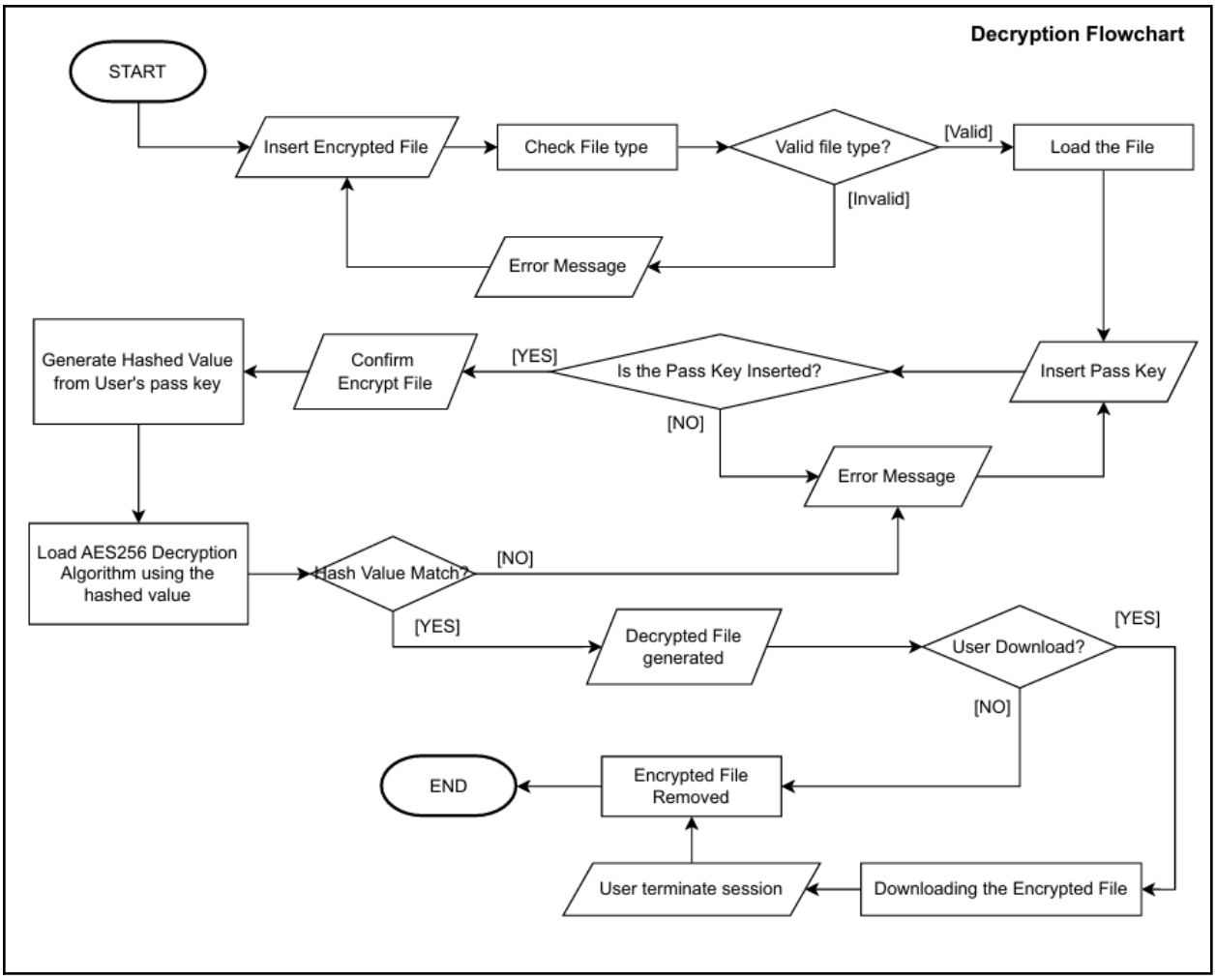


Figure 4.3: Flowchart diagram for decryption process.

The process for decryption is kinda similar to the encryption process but just in reverse. The encrypted file will be decrypted using a hash value(encryption key) generated from the user's pass key. The AES256 algorithm will be initiated first in the decryption process, only then it can have a check whether the encryption key used is correct or incorrect. If the user's passkey is first incorrect, then the generated hash value is definitely wrong and can't match with the AES256 algorithm in decrypting the encrypted file. However, if the passkey is correct, then it vice versa.

4.1.3 Activity Diagram

Activity diagram is used to visualize the flow of control or data from activity to activity within a system, emphasizing the sequence and conditions for coordinating lower-level behaviors.

A. Activity Diagram for Encryption process

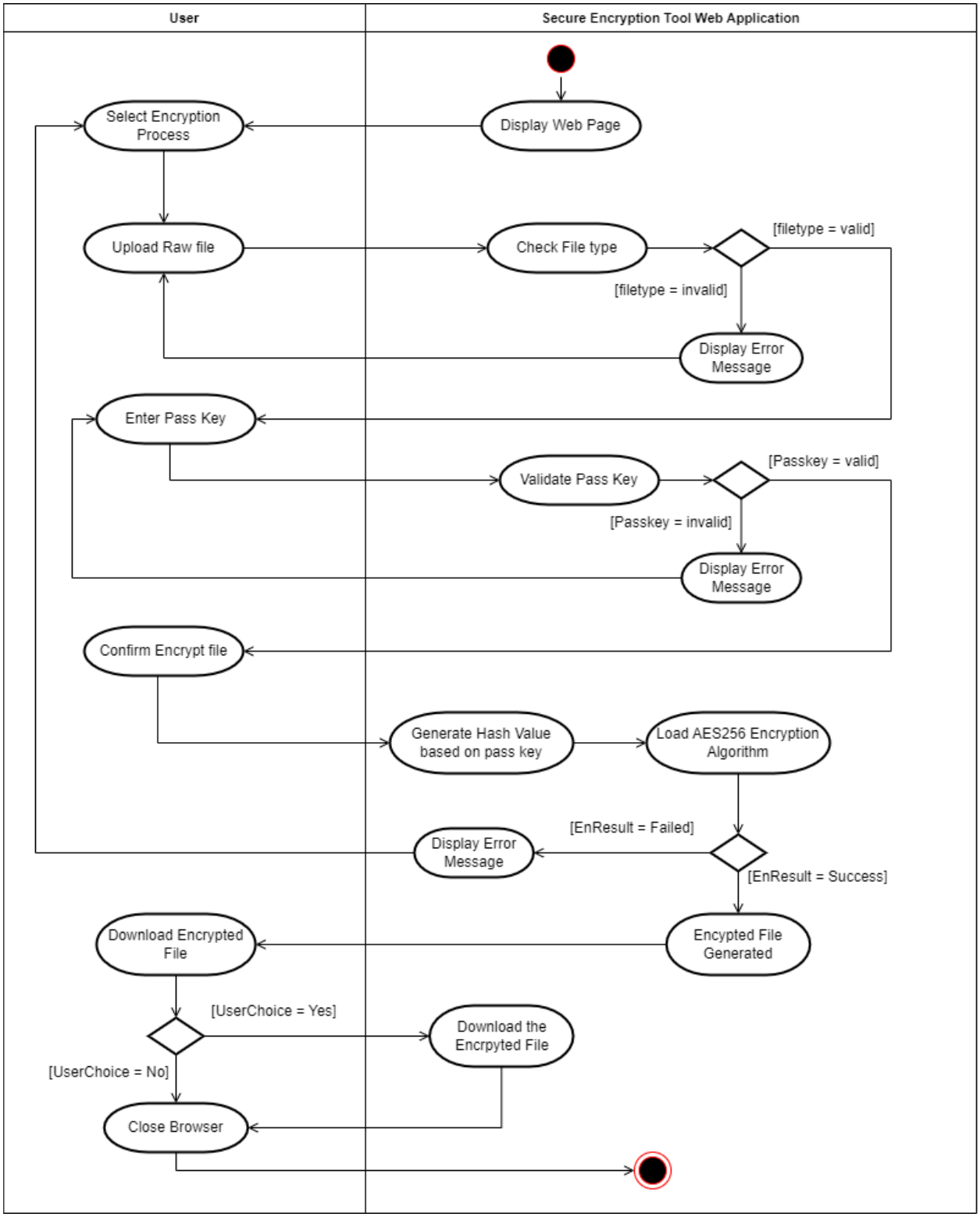


Figure 4.4:Activity diagram for the encryption process

The content and explanation for the activity diagram of the encryption process is quite similar to the content explanation in Figure 4.2. There are 4 decision point that could occur in the process and each point also represents validation points. At first, if the inserted file is not supported, the user will be prompted to re-insert until it is a valid file type. The passkey cannot be left empty and users must insert a pass key of their own and only be remembered by themselves. The process of encryption has the potential of encountering error and if that happens an error message will be displayed and redirect the user to re-perform the whole encryption process. Once the encrypted file is successfully generated, it is up to the user;s choice to download it as in the end, no file will be saved into the web app.

B. Activity Diagram for Decryption process

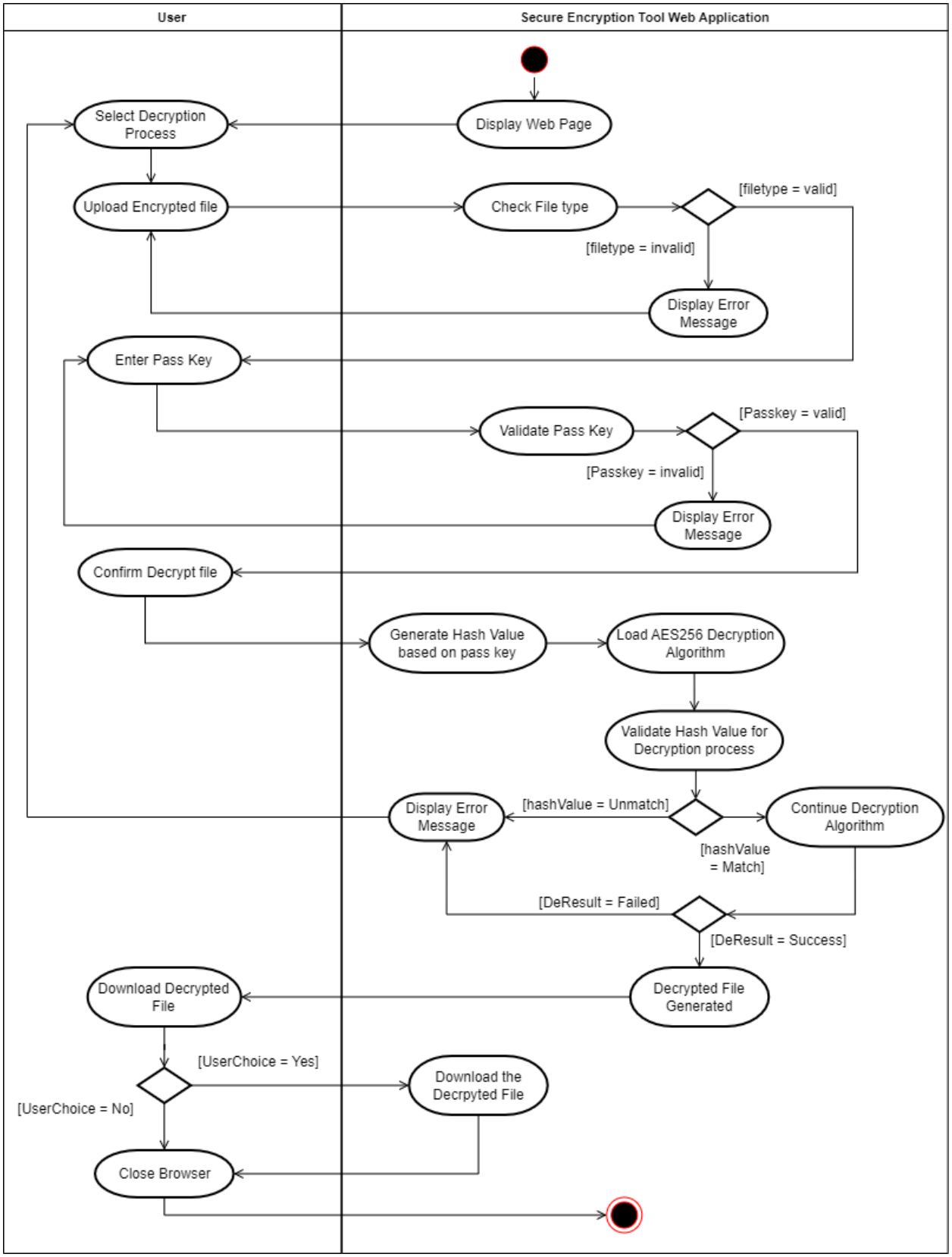


Figure 4.5: Activity Diagram for Decryption process

The content and explanation for the activity diagram of the decryption process is quite similar to the content explanation of the activity diagram of the encryption process.

4.1.4 Sequence Diagram

Sequence diagram details how objects interact in a particular scenario of a use case, focusing on the order of messages exchanged between objects over time to carry out a function.

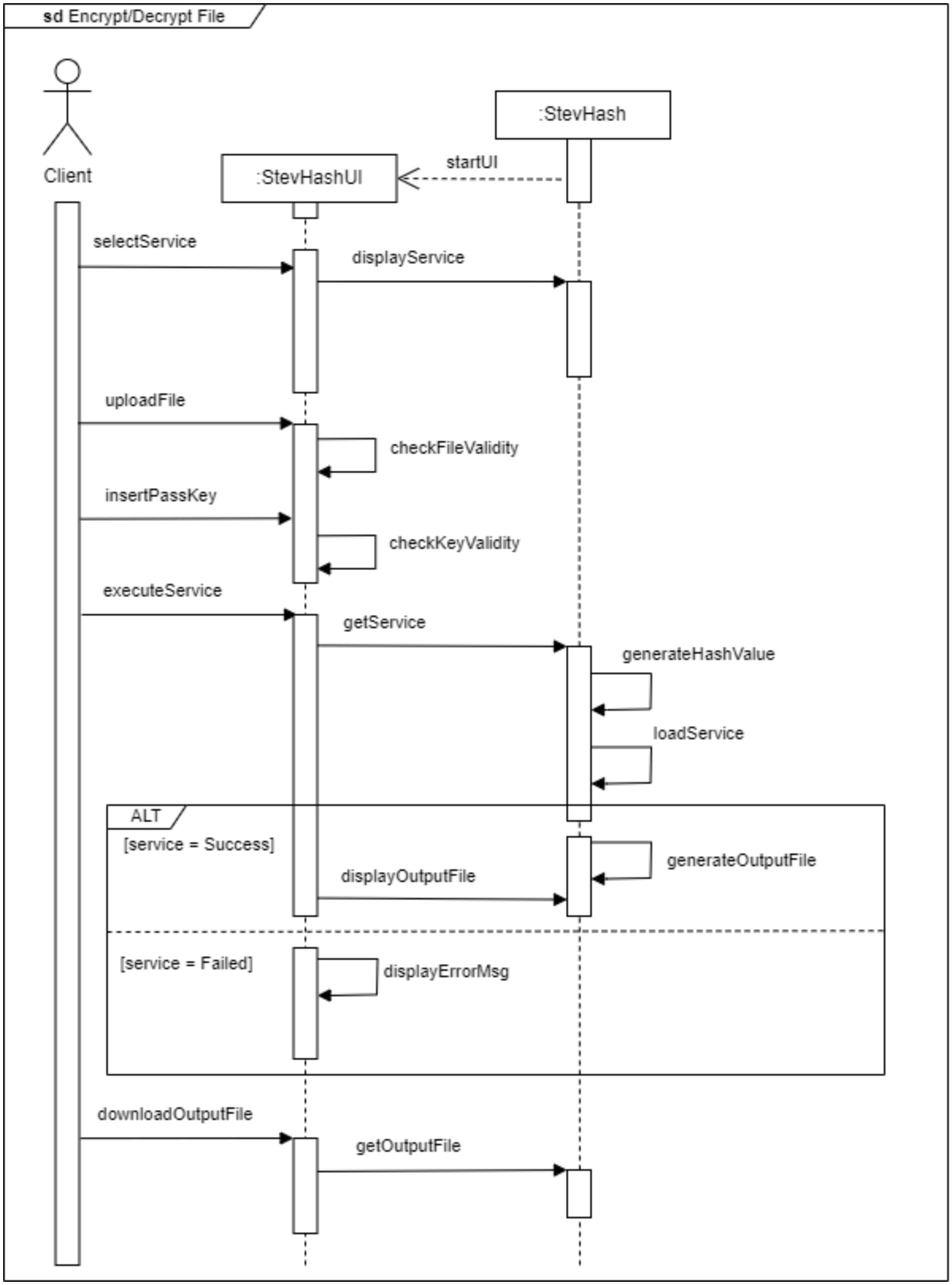


Figure 4.6: Sequence diagram for encryption/decryption service.

In the activity diagram, it shows no other objects but just :StevHashUI and :StevHash because this is a client-side encryption service when everything is done on the client's browser device. Thus, there will be no persistent object being created as well as the requirement of persistent database storage to store/retrieve them. The “service” in the diagram refers to encryption or decryption service, depending on the user's choice. The method “loadService” refers to the loading AES algorithm for the encryption/decryption process after getting the hash value.

At first, users can select a service at the front end of the web app. The web app will refer to it’s controller (:StevHash) to display available services which in this case are the encryption or decryption services. Once the user selects the service, the user has to insert a file and pass a key. The validation of file type and pass key will be processed first without need to access the controller. Once every input is valid, the user can execute the service. The execution of service will require the front end UI to interact with the controller to perform backend’s tasks which are the generation of hash value and the execution of AES256 algorithm. If the service is finished successfully, an output file (encrypted or decrypted file) will be generated and displayed to the user to download it. Alternatively, if the service failed to execute successfully, an error message will be displayed and redirect the user to re-perform the whole process. This sequence is similar to the flow in the activity diagram and flowchart diagram.

4.1.5 Deployment Diagram

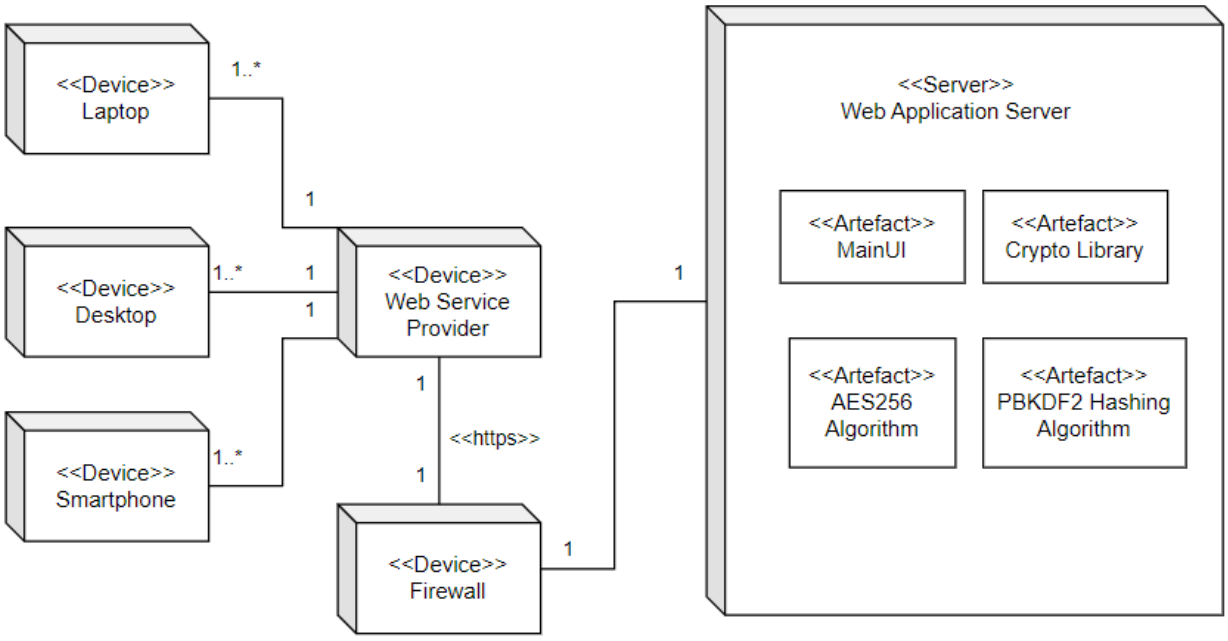


Figure 4.7: Sequence diagram for encryption/decryption service.

Even though the StevHash web app is a client-side encryption process where encryption happens on client-side, the application itself still needs to be served from somewhere. So in order to host the web application files and serve them to the client, StevHash web app is to be deployed on the server.

In the deployment diagram, any device such as laptop or desktop or smartphone as long as it is a web-service/browser supported device, they can access to the StevHash. Each device will access 1 web service provider. The web service will direct users through a secure channel that applies Hypertext Transfer Protocol Secure (HTTPS) to reach the firewall first for total security before gaining access to the StevHash that is located on the web server. The artifacts that will be included in the StevHash web app are basically main UI for user to interact with the application, AES256 algorithm for encryption/decryption services, PBKDF2 hashing algorithm for hash value generation and Crypto library for more complex encryption/decryption process.

4.1.6 Class Diagram

Class diagram describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. Do take note that the class diagram display here is for concept system design which acts as a reference only for final web app design.

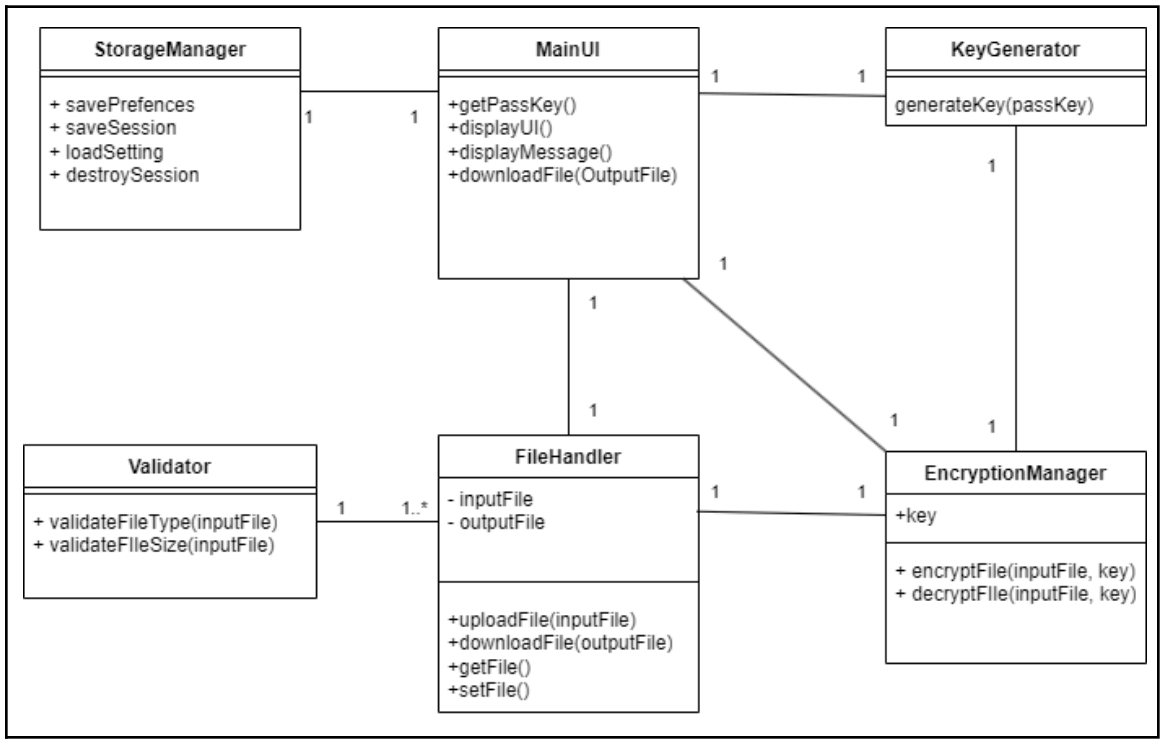


Figure 4.8: Class diagram of StevHash web application.

The class diagram for StevHash web application consists of five main classes: UserInterface, FileHandler, EncryptionManager, KeyGenerator, and Validator. Each class has specific roles and interacts with others to facilitate the secure encryption and decryption of files directly within the user's browser.

UserInterface class generally manages all interactions with the user, collects the pass key from the user, displays messages and provides the option to download the encrypted or decrypted file. Meanwhile, FileHandler class handles file operations, manages the uploading of the raw file (inputFile) and the storage of the resulting file (outputFile) after encryption or decryption. FileHandler class also ensures files are appropriately handled for download by the user. On the other hand, the class EncryptionManager act as the central to processing the encryption and decryption. EncryptionManager uses a key generated by KeyGenerator to encrypt or decrypt the

inputFile, producing an outputFile. The class KeyGenerator is required to generate a hash value from the user-provided pass key. This hash value is used as the encryption key by EncryptionManager. Other than that, the class Validator ensures that the file type and size of the inputFile are supported and within acceptable limits before processing.

The interactions between classes are explained here.The UserInterface initiates the file encryption or decryption process by interacting with FileHandler to upload the file and with KeyGenerator to generate the necessary encryption key. Validator is used to check the file's validity before any encryption or decryption process begins. The EncryptionManager takes the validated file and the key to perform the required operation, either encrypting or decrypting the file. Once the process is completed, the UserInterface provides the user with the option to download the resulting outputFile.

The class StorageManager is still relevant despite there not being a requirement of a permanent database. StorageManager is to handle local storage operations, such as saving user preferences or UI settings that aren't sensitive but improve user experience. For example, we might want to remember the user’s preferred selection of service, which does not compromise security.StorageManager can also be utilized as Web Storage (localStorage or sessionStorage) depending on the need to retain settings across sessions (localStorage) or just for a single session (sessionStorage). Not just that, StorageManager can manage temporary data that might need to be stored during an active session. This can include non-sensitive operational data that helps maintain the state or performance of the application without impacting security.

All data attributes within the classes are kept private to ensure data integrity and security, only accessible through public methods designed to handle specific tasks safely and efficiently.

4.2 Database Design

The project development of the secure encryption tools web application of StevHash does not provide the needs for database design. The StevHash web app would only need a Session table to manage active sessions.

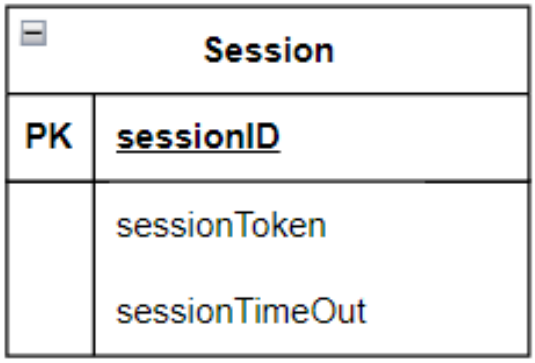


Figure 4.9: Session database table.

With this design, when a user accesses your web application, a session record is created with a unique session token and an expiration time. This session token is then used for subsequent requests to identify the user's session. The expiration time ensures that inactive sessions are cleaned up regularly to maintain system performance and security.

In this setup, the StevHash web application operates entirely in the user's browser without the need for server interaction after the initial delivery of the web page. Here's why a database is often not used:

a. No Server-side Data Handling: Since all encryption and decryption tasks are performed locally, there is no need to store encrypted files, keys, or hashes on a server.
b. Security and Privacy: By not using a server-side database, the application enhances security and privacy. No sensitive data is at risk of being exposed if a server breach occurs.
c. Zero-Knowledge Principle: By not storing passwords or even their hashes on the server, your application adheres to the zero-knowledge principle, where the service provider knows nothing about the user's data or encryption keys.

This client-side processing model ensures that the application is secure and respects user privacy, crucial for applications dealing with sensitive data.

4.2.1 Understanding Client-Side Encryption

Client-side encryption means that all encryption and decryption processes are carried out on the user's device, specifically within their browser. This approach ensures that sensitive data, such as files and passwords, do not leave the user's device in an unencrypted state.

Example of Client-Side Encryption Web Application flow:

1. User Uploads a Raw File: The user selects a file and uploads it. Despite saying "upload," in client-side encryption, this file does not actually go to a server. Instead, it is read directly from the user's file system by the browser.
2. User Enters a Pass Key: The user types in a pass key. This key is used locally within the browser and is not sent to any server.
3. Generation of Hash Value: The web app generates a hash of the pass key (still within the user's browser). This hash value is used as the encryption key for the AES256 algorithm but does not leave the user's device or browser.
4. Encryption Process: The web app encrypts the file using AES256 with the hash value as the key. All of this processing happens in the user's browser.
5. User Downloads the Encrypted File: The now encrypted file is available for the user to download directly to their device. No encrypted file is sent to or stored on the server.
6. Decryption Process: To decrypt, the user must provide the original pass key again locally, which the app hashes to recreate the encryption key. The decryption process also occurs entirely in the browser.

4.2.2 Real-Scenario Handling of user's pass key

Assume that the user encrypt file on Tuesday, then decrypt it on Saturday

1. Hash Value Generation: When a user decides to encrypt a file, they input a passkey. In the browser, this pass key is hashed using a secure hashing algorithm. This hash serves as the encryption key for the AES256 algorithm to encrypt the file.
2. Encryption Process: The hash (encryption key) is used to encrypt the file locally in the browser. The original pass key is neither stored or transmitted; only the hash is used for the encryption process.

3. Storage of the Hash Value. Because of the zero-knowledge principle, the web app will ideally not store the hash value on any server or persistent storage to ensure that only the user can decrypt the file.
4. However, the hash value can be stored temporarily in the browser's session storage (Local Temporary Storage) if the user needs to perform multiple encryptions or actions within the same session. However, session storage is cleared when the browser session ends, and it is not accessible across different sessions or days.
5. No Persistent Storage: The hash value is not persistently stored anywhere. This means that if the user closes the browser or the session ends, they will need to regenerate the hash by entering the pass key again when they return to decrypt the file.

Decryption Process:

User Re-entry of Pass Key: On Saturday, when the user returns to decrypt their file, they must enter the same Pass key to be able to generate the same hash value.

4.3 User Interface Design

User Interface(UI) design served as the front-end interaction between user and the application. The elements of UI design are important in ensuring that the user has the best use of the developed application. This project will cover the UI design in the aspect of wireframe and prototype design.

4.3.1 Wireframe

A wireframe is designed to layout the necessary content and functionality of the StevHash web application in order to enhance user needs and experiences in using the web app.

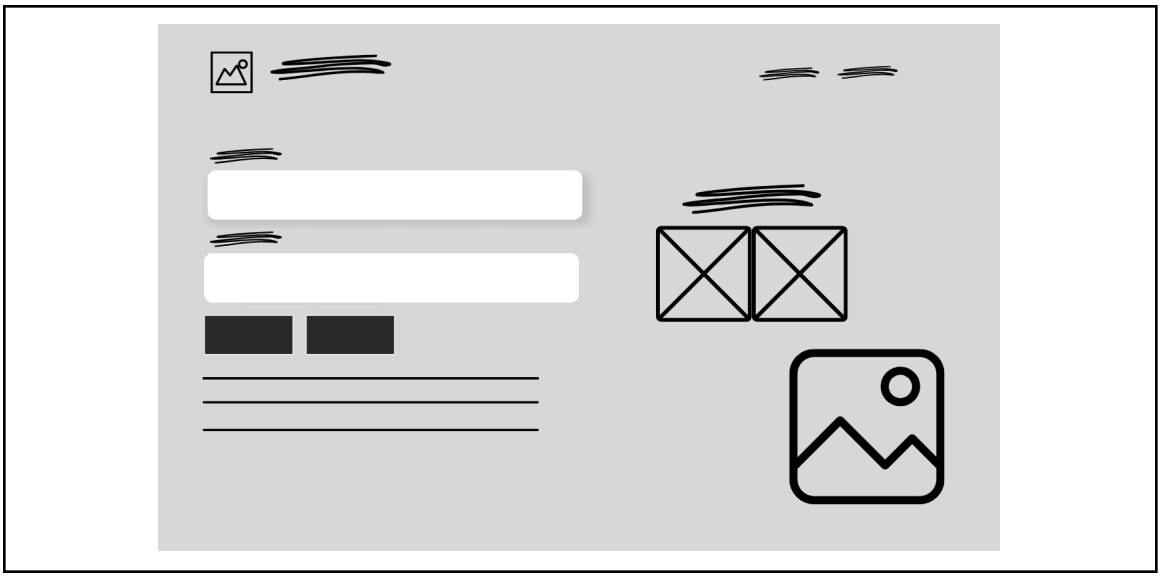


Figure 4.10: Layout for Home page

This will be the basic layout for the homepage. The basic criteria designed for this homepage consisted of a few logos and attractive images that serve as the attention seeking & cognitive recognition for users to know they are in the IT security-related environment. There should be 2 input parameters here to receive the user's uploaded file and pass key. At least 2 buttons will be implemented here to allow encrypt and decrypt action.

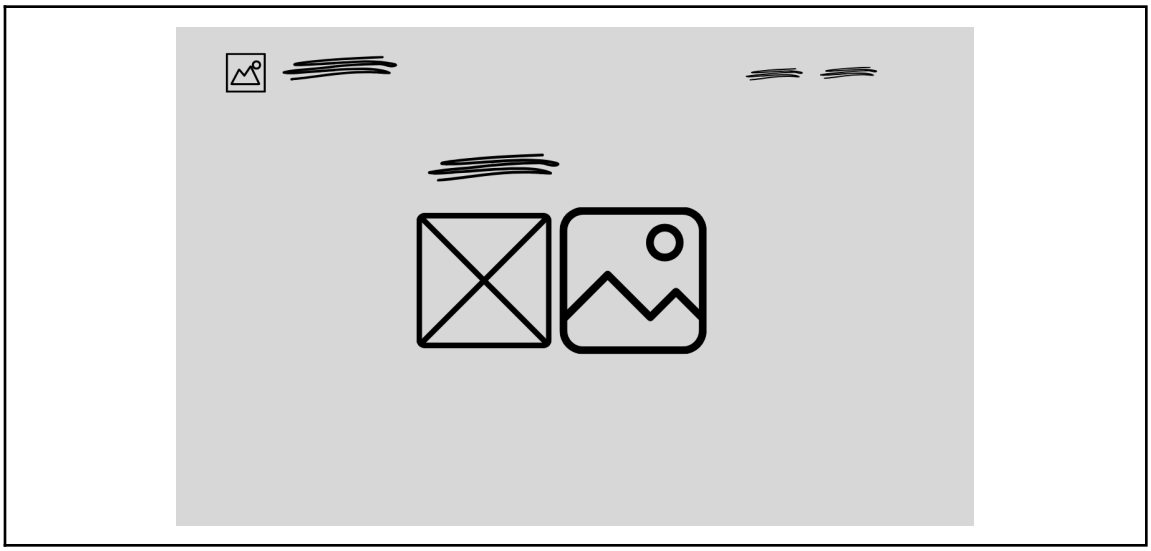


Figure 4.11: Layout for processing display

This will be the layout for the display of the processing of encryption or decryption. An interactive and moving image is required here to allow the user to have an ease of feeling that the program is running.

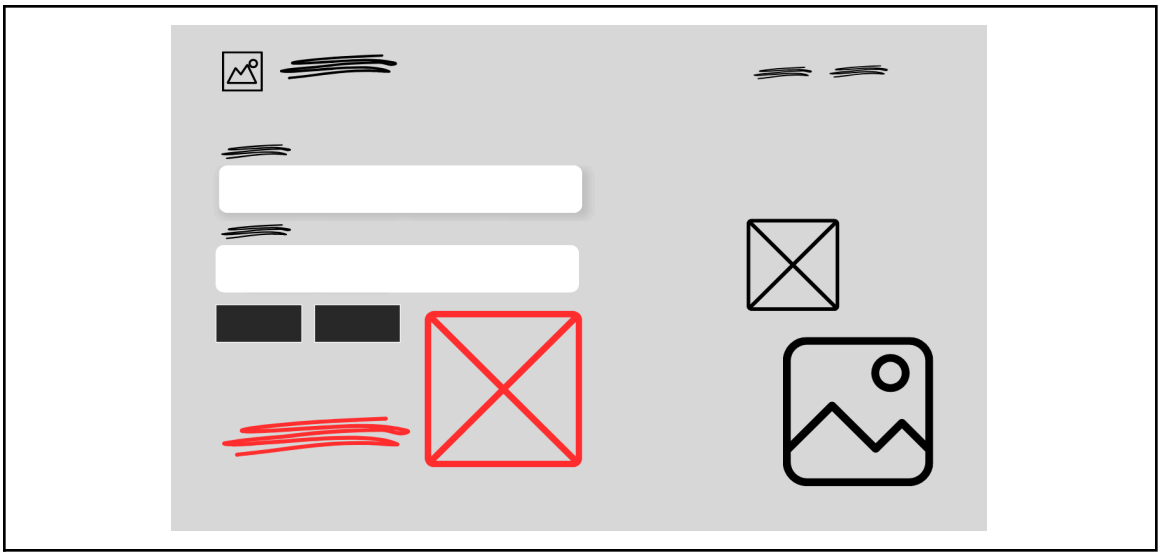


Figure 4.12: Layout for unsuccessful service

This will be the layout of the display of failed encryption/decryption results. The layout design is similar to the homepage but with added red-color image and wordings here. Those criterias are important to seek the user's attention and understand what error is occurring.

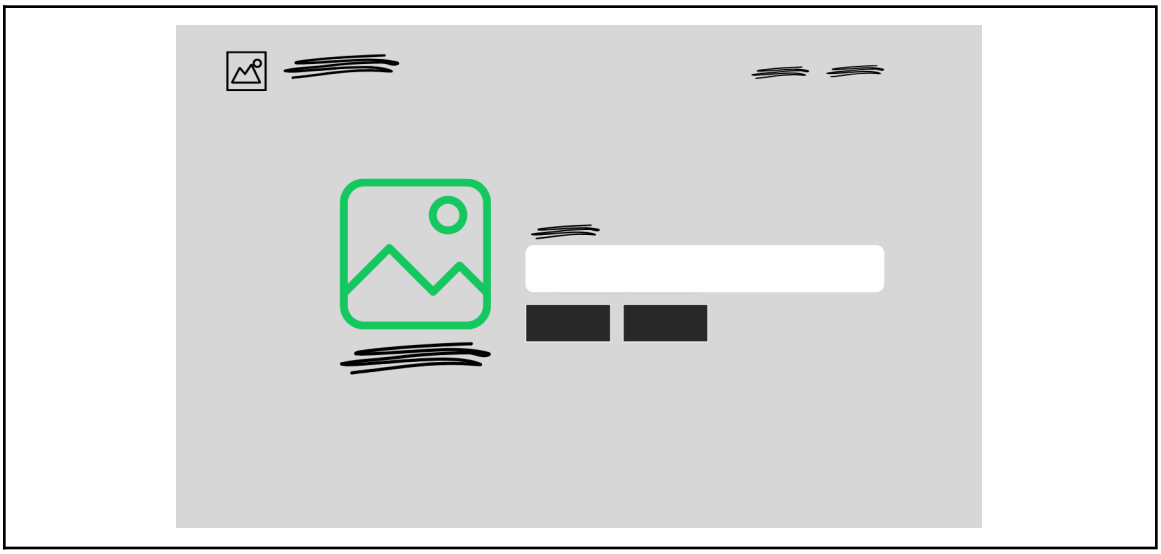


Figure 4.13: Layout for successful service

This will be the layout of the display of successful encryption/decryption results. The layout required an image icon to serve as a direct message to let users know immediately that the result is success. 1 display bar is required to display the generated output file. 2 buttons are required in this layout to ensure the user can have the download options or back to the homepage.

4.3.2 Prototype Design

Prototype design is used to test the practicability of the wireframe design, Below is the display of prototype design for each web page based on the wireframe.

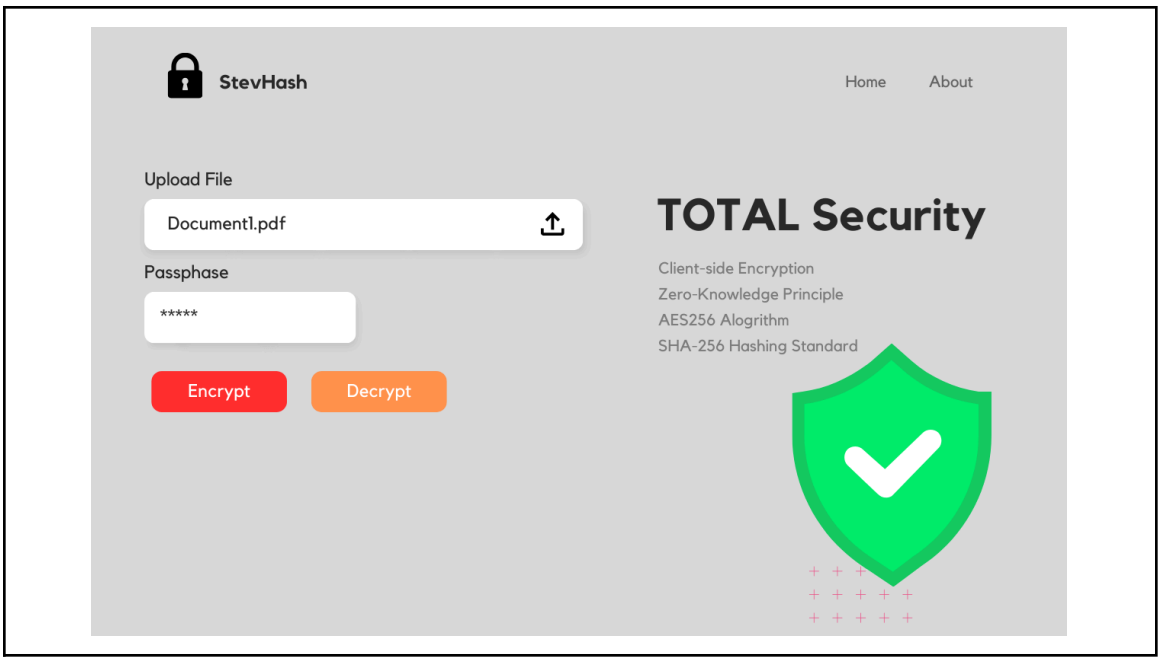


Figure 4.14: Prototype design for home page

This is the home page, allowing users to insert files and pass keys first. After that, the user can click the action button. The color elements used here are based on attention-seeking criteria. The green color is to ensure users feel safe using our web app. The background light-gray color gives a sense of calmness. “Encrypt” button is to encourage users to encrypt a file first before they can decrypt the file. The pass key is disclosed to enhance security.

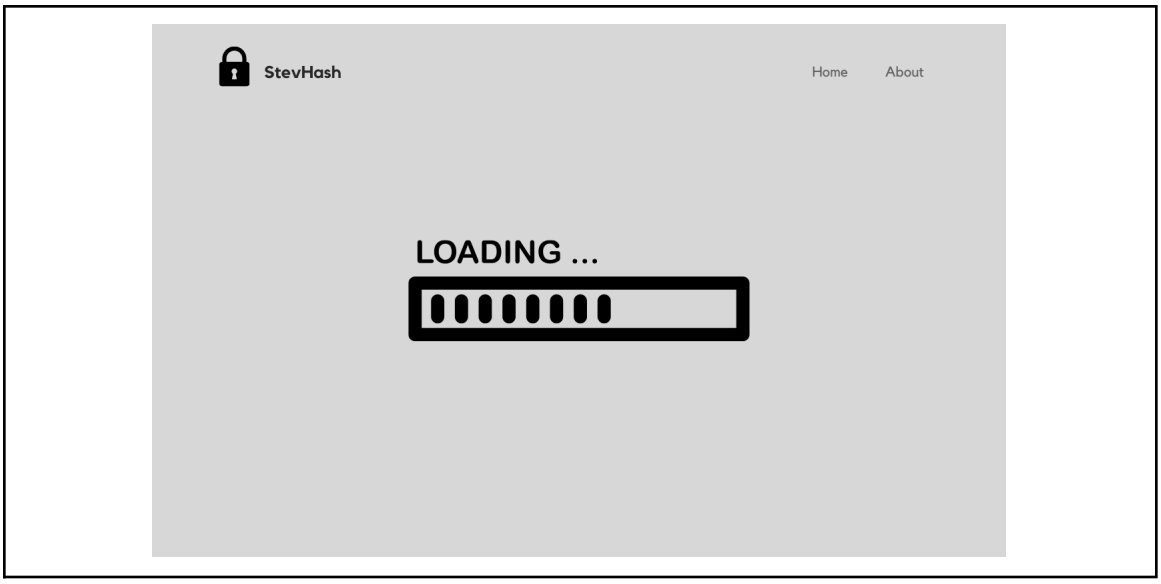


Figure 4.15: Prototype design for processing display

The encryption/decryption process will be shown like this. The image is interactive, giving users a more confident feeling as they will know that the web app is still running.

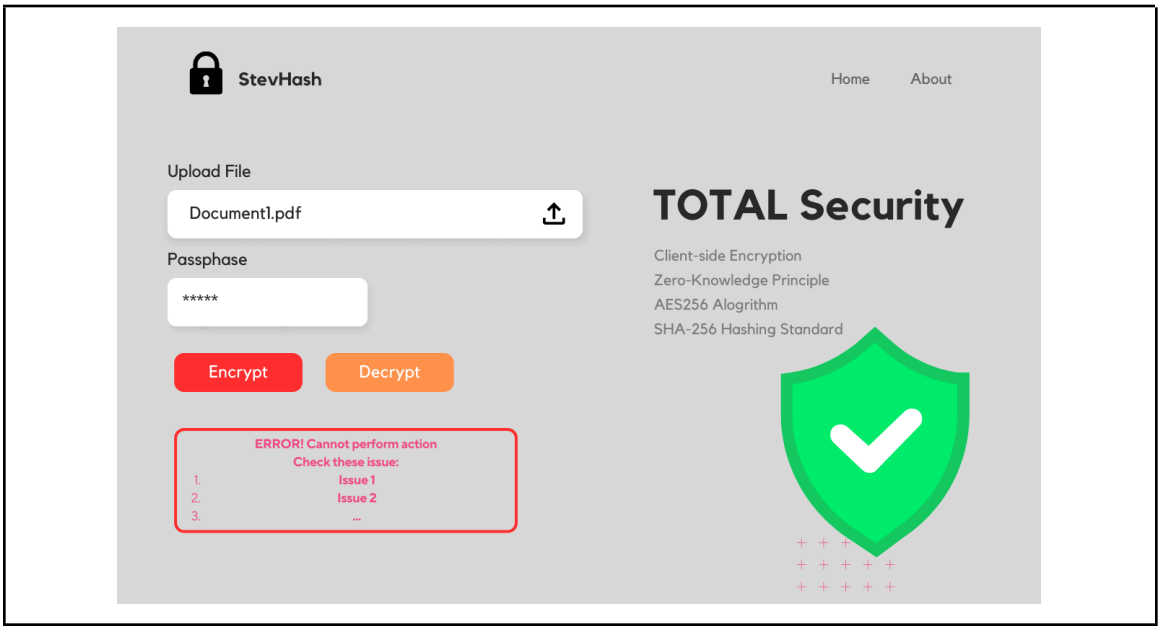


Figure 4.16: Prototype design for failed service.

If there's an error in the process, the user will be redirected back to the home page, with the error message being displayed on the homepage. Red-color error messages will catch the attention of users to look into the error more seriously.

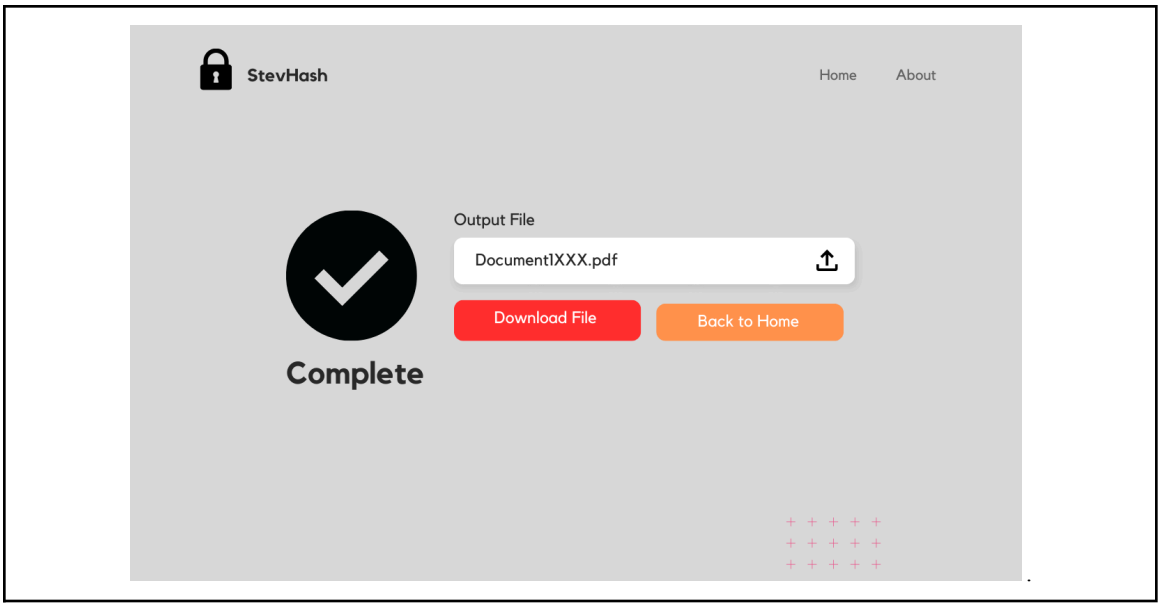


Figure 4.17: Prototype design for failed service

If the encryption/decryption process is successful, the output file will be generated, and allow the user to download it if desired. The success of the encryption/decryption process will be presented with a big “complete” icon, letting users easier and faster to learn that the process has finished. The red-colored button encourages users to download their resulting output files, avoiding them to forget from downloading it.

4.4 Chapter Summary and Evaluation

Chapter 4 of the FYP delves into the intricate system design of the StevHash secure file encryption tool, elucidating its architecture, user interactions, and operational processes. Through various diagrams including flowcharts, activity diagrams, and sequence diagrams, the chapter outlines the step-by-step processes involved in file encryption and decryption, ensuring a seamless and error-free user experience. The deployment diagram illustrates the application's hosting and accessibility, while the class diagram provides insight into the structural composition of the system. Notably, the absence of a traditional database underscores the application's emphasis on security, privacy, and adherence to zero-knowledge principles. Overall, Chapter 4 offers a comprehensive understanding of the StevHash system, its functionalities, and design considerations.

Chapter 5

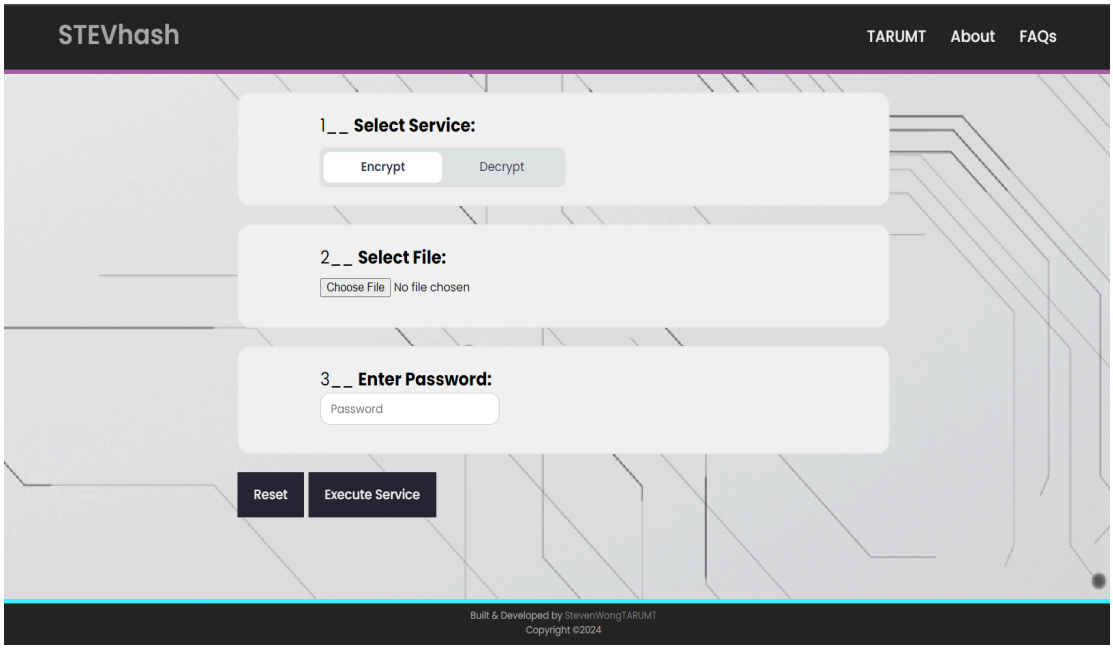
Implementation and Testing

5 Implementation and Testing

This chapter provides an overview of the implementation and testing of the "StevHash" web application, a secure platform for file encryption and decryption using AES-256 encryption. It covers the architecture and core components of the system, including the encryption and decryption processes, user interface design, and server setup. Additionally, it describes the error handling mechanisms and security measures implemented to protect user data. The chapter concludes with a discussion of the testing strategies employed, including unit, integration, system, and user acceptance testing, to validate the functionality, security, and user experience of the application.

5.1 Overview of System Implementation

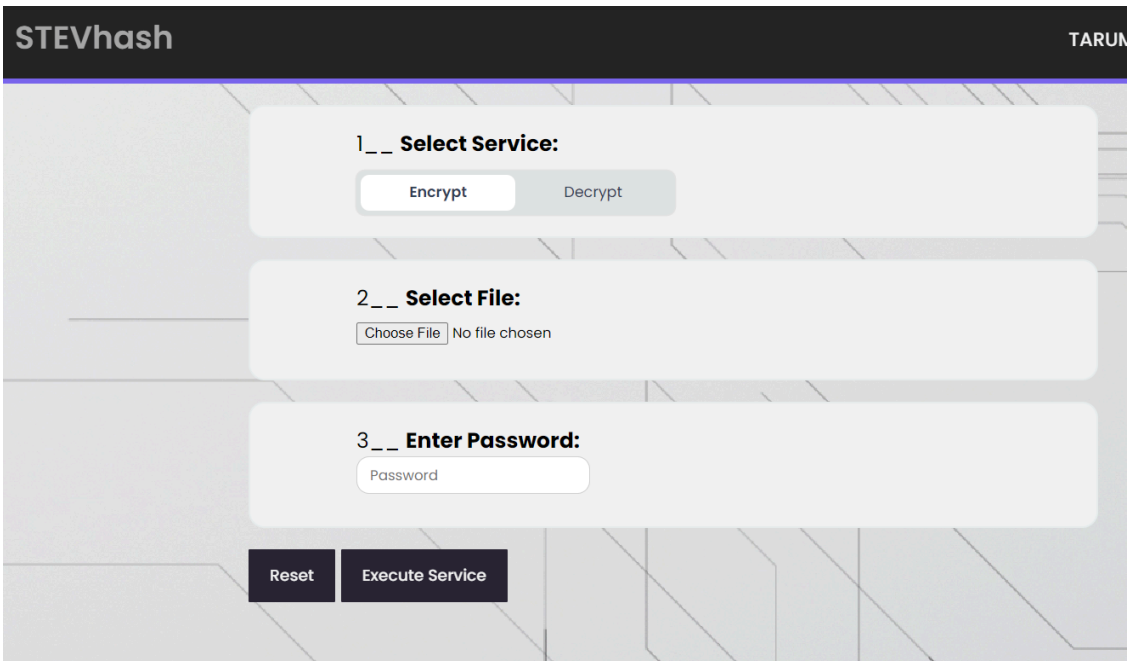
The implementation of StevHash involved the development of a secure web application designed for encrypting and decrypting files using the AES-256 encryption standard. The application adheres to the zero-knowledge principle by not storing users' passkeys. Instead, a derived key from the user's input passkey, combined with a salt, is used to perform encryption and decryption operations. The system was built using Python with the Flask framework for the backend, and HTML, CSS, and JavaScript for the front end.



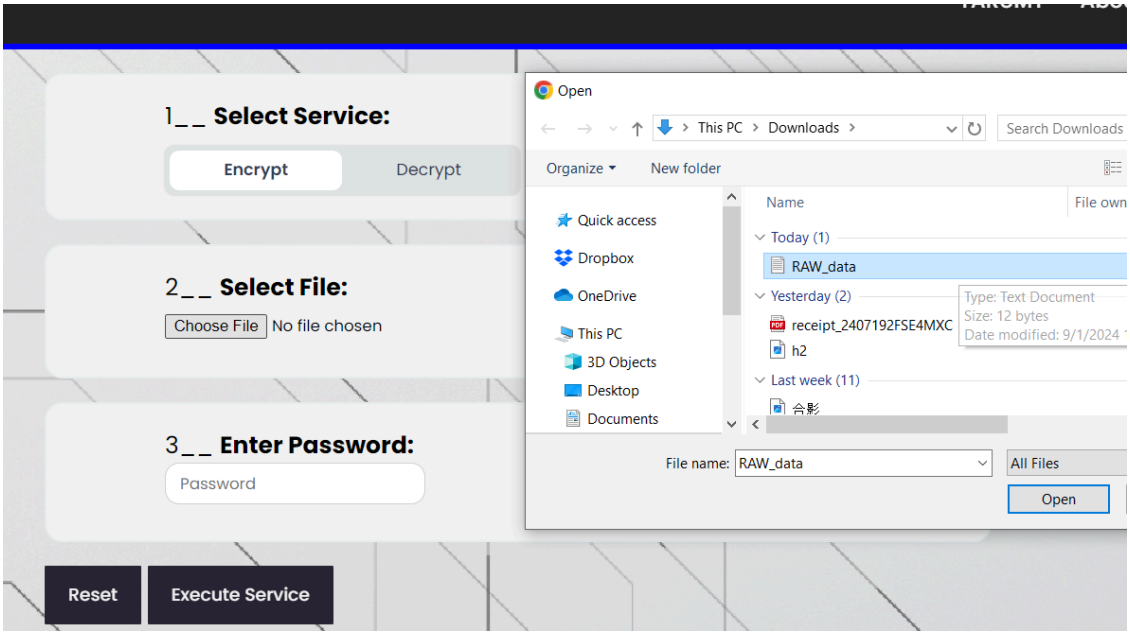
The following sections detail the core components of the system, including the encryption and decryption processes, key derivation, user interface design, and server settings.

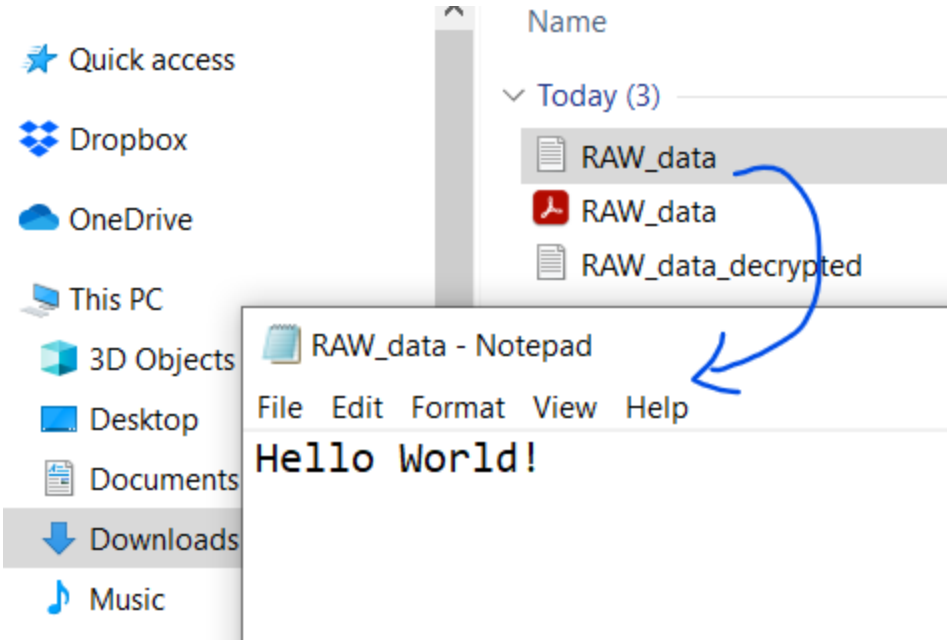
5.1.1 System Usage

Below displays the process of how a user can use the final developed web application named “StevHash” to securely perform file encryption/decryption service.

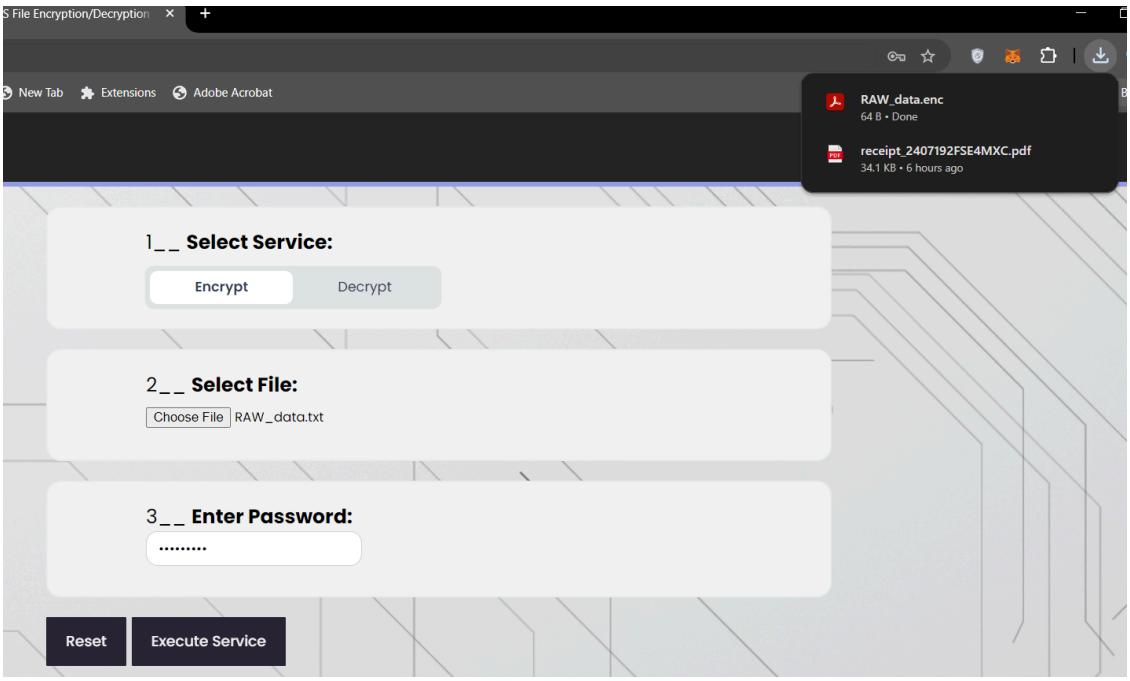


At first, the user is required to select a service(encrypt/decrypt). Let’s start with encryption.

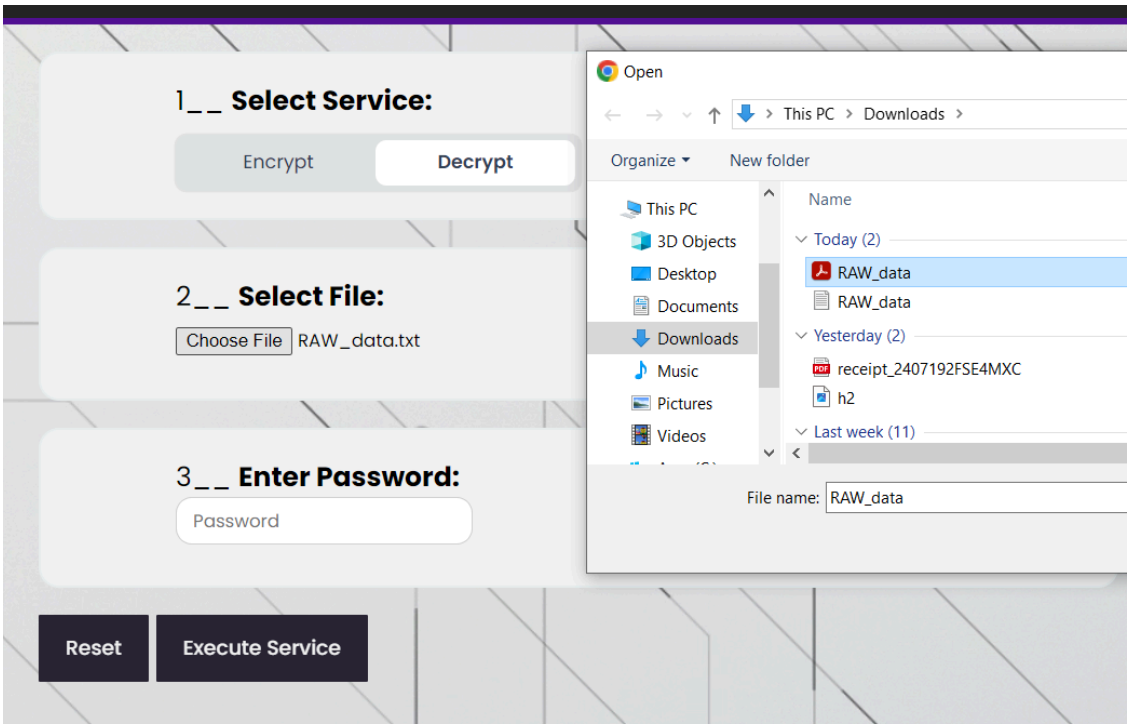




Then, the user has to upload a raw file for encryption. Assume that the user uploaded a text file named “RAW_data” with the text content of “Hello World!” in it.

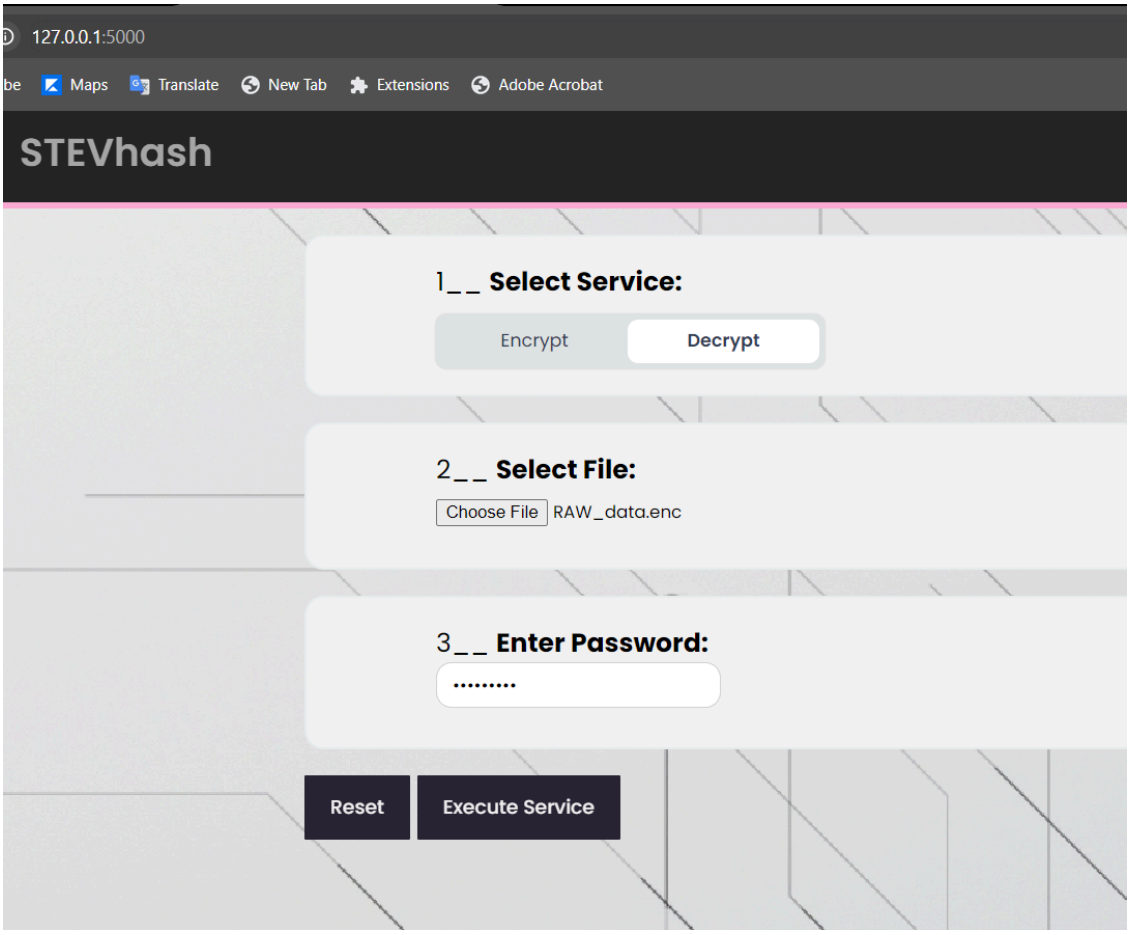


Then, the user have to upload a file and insert a password (eg. dUmb@557!) and then click on the “Execute Service” button. The encryption process will be executed and return the resulting encrypted file to the user. The web application itself will auto-download the resulting file to the user. In this case, the resulting file is called RAW_data.enc in enc format.

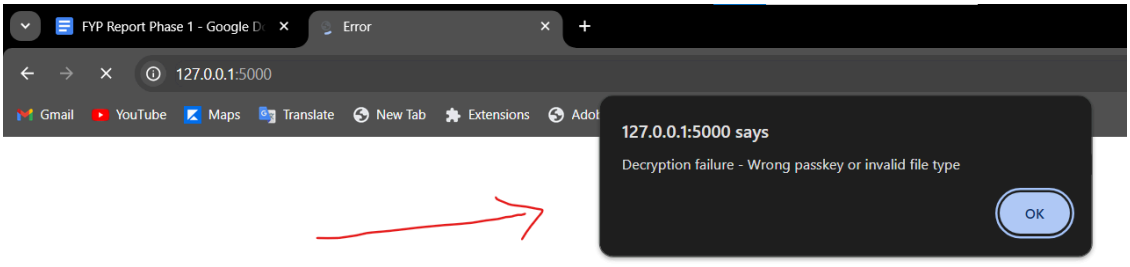


When the user wishes to open the RAW_data.enc, the content is no longer available to be accessed nor viewed.

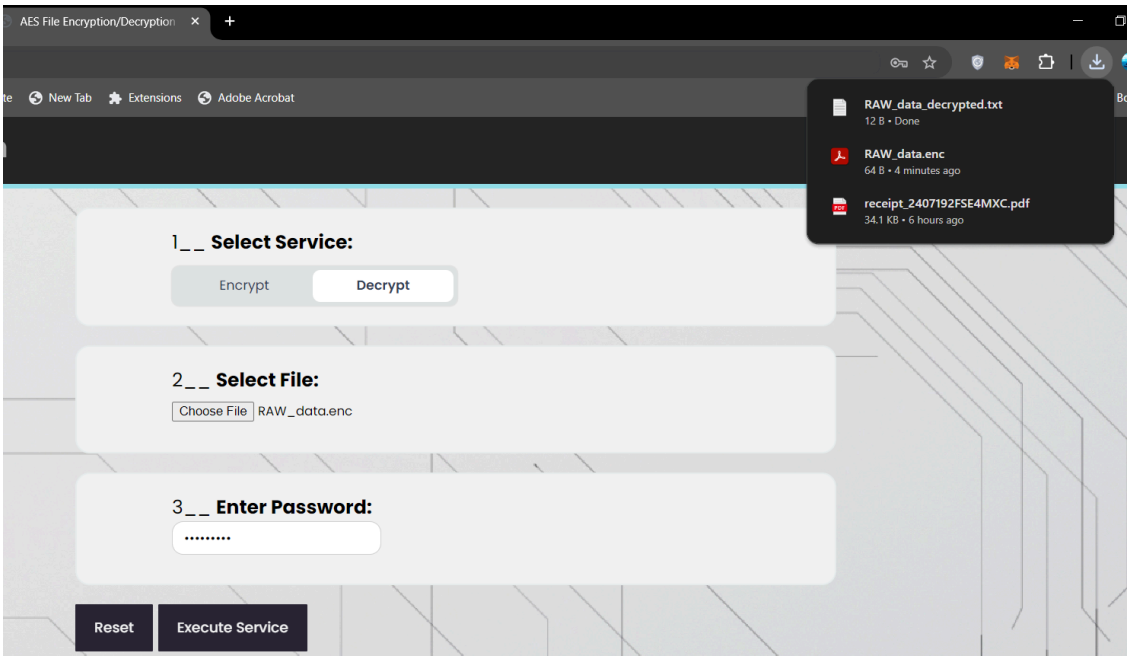
Now, let's repeat everything again for the decryption process. As usual, we repeat the same process but we choose "decrypt" service.



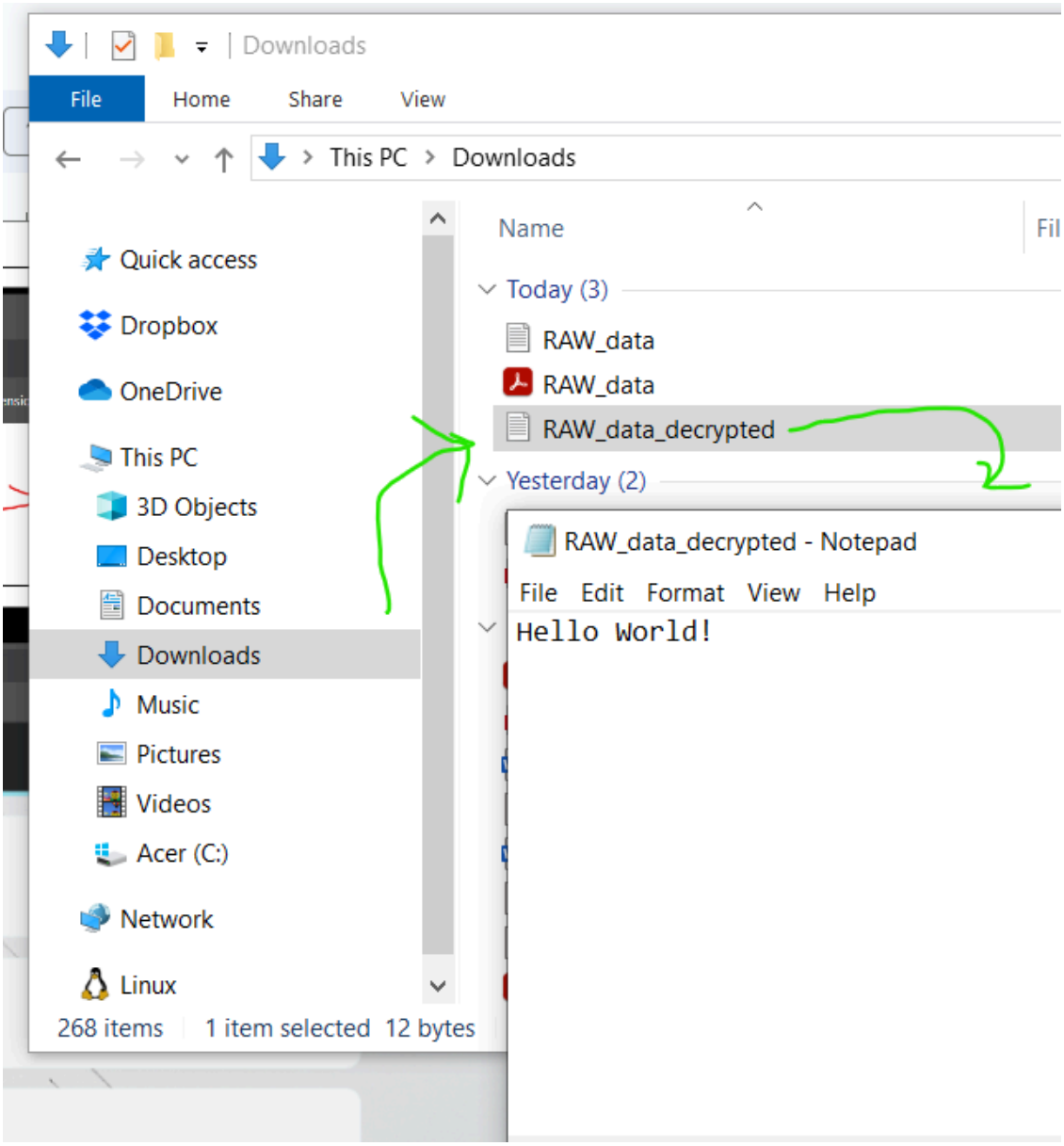
Then we uploaded the encrypted file which is RAW_data.enc.
Let's assume that we key in the wrong password for decryption.(eg. abc123)



The web application will then display a dialog box showing an error message and redirect users back to the main page again when the user clicks "OK".



Now let's try that again but with the correct password (dUmb@557!). The decryption process is successful and the resulting file named "RAW_data_decrypted.txt" is generated in its original format.



The user can go to “Download” to view the resulting decrypted file and verify its content. The content is still the same as before encryption which displays the text “Hello World!”.

5.2 Front-End Implementation

The frontend of StevHash is designed using HTML, CSS, and JavaScript, providing a simple and user-friendly interface for uploading files and selecting encryption or decryption services.

5.2.1 User Interface(UI) code snippets

The UI is designed to be simple and user-friendly, allowing users to choose either encrypt or decrypt the file, then upload files, enter a passkey, and finally execute the selected service. The HTML code handles the UI interactions and ensures that the user input is correctly formatted before sending it to the server. All the input fields are implemented with the "required" keyword to avoid users from performing empty input.

```
<div class = box1>
    1__
    <label style="font-weight: bold;">Select Service:</label><br>

        <div class="radio-inputs">
            <label class="radio">
                <input type="radio" name="service" value="Encrypt" required>
                <span class="name">Encrypt</span>
            </label>

            <label class="radio">
            <input type="radio" name="service" value="Decrypt" required>
            <span class="name">Decrypt</span>
            </label>
        </div>

</div>
```

Figure 5.1: From design with Input field for user to select service(encrypt/decrypt)

```
    <div class = box1>
    2__
    <label style="font-weight: bold;">Select File:</label><br>
    <input type="file" name="file" required><br><br>
    </div>

    <div class = box1>
    3__
    <label style="font-weight: bold;">Enter Password:</label><br>
    <input type="password" name="password" placeholder="Password" class="input" required><br><br>
    </div>

    <div class = box2>
    <input type="reset" value="Reset" onclick="resetForm()" class="buttonA">
    <input type="submit" value="Execute Service" class="buttonA">
    </div>
</form>
```

Figure 5.2: Form design Input field for user to upload file, insert password and execute service

Based on figure 5.2, the division `class = box2` consisted of two buttons which are the button for reset and button for executing selected service from the previous input field. The reset button will call the `resetFrom()` function that clears all the input fields of the web application back to its original default form, which is all empty. The submit button with value of "Execute Service" will receive all the current content from all input fields and send it to the backend source code for encryption/decryption process.The form data is sent to the backend for processing via a POST request.

5.2.2 Error Handling and User Feedback

To ensure a smooth user experience, the system provides feedback through Flask's flash() messaging and JavaScript alerts for errors. This can be shown on a decryption error (e.g., incorrect passkey), the system will display an alert message to the user and redirect back to the main page.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Error</title>
    <script>
        function showError() {
            alert("{{ error_message }}");
            window.location.href = "{{ url_for('index') }}";
        }
    </script>
</head>
<body onload="showError()">
</body>
</html>
```

Figure 5.3: Source code for error.html consisting of Javascript code to handle error.

5.3 Core Implementation Components

The core functionality of StevHash revolves around secure encryption and decryption of user-uploaded files. The encryption and decryption are handled by the Python Crypto library (PyCryptodome), which provides robust cryptographic algorithms. These processes are also being implemented using the Advanced Encryption Standard (AES) with a key size of 256 bits, which is widely regarded as one of the most secure encryption methods. The core implementation of this project resides within the AESservice.py source code.

5.3.1 Encryption Process Implementation

The encryption process involves converting the original file data into an encrypted format that is unreadable without the correct decryption key. Here's a step-by-step breakdown of how the encryption process is implemented:

```python
def encrypt_file(file_data, key, salt, file_extension):
    """Encrypt the file data in memory, including the original file extension."""
    cipher = AES.new(key, AES.MODE_CBC)
    iv = cipher.iv

    try:
        # Store the extension at the beginning of the encrypted file
        extension_bytes = file_extension.encode('utf-8').ljust(16)
        ciphertext = cipher.encrypt(pad(extension_bytes + file_data, AES.block_size))
        encrypted_data = salt + iv + ciphertext
        return io.BytesIO(encrypted_data)
    except Exception as e:
        return str(e)
```

Figure 5.4: Code snippet for encryption process.

STEP 1: Key Derivation using PBKDF2

The first step in the encryption process is to derive a strong cryptographic key from the user-provided passkey. This is achieved using the Password-Based Key Derivation Function 2 (PBKDF2). PBKDF2 takes the user's passkey and a randomly generated salt and produces a 32-byte (256-bit) key suitable for AES-256 encryption. The salt is used to ensure that even if two users choose the same password, their derived keys will be different. PBKDF2 is generally more preferable than SHA-256 for key derivation because PBKDF2 is specifically designed to be computationally intensive and slow, using multiple iterations (e.g., 1,000,000) and a salt to generate a cryptographic key. This makes it highly resistant to brute-force and dictionary attacks. In contrast, SHA-256 is a fast cryptographic hash function, which makes it vulnerable to rapid brute-force attacks if used directly for password hashing or key derivation.

```
def get_key(password, salt):
    """Generate a key from the password using PBKDF2."""
    key = PBKDF2(password, salt, dkLen=32, count=1000000)  # AES256 key length is 32 bytes
    return key
```

Figure 5.5: Code snippets for key derivation using PBKDF2

- Salt Generation:
  A salt is a random 16-byte value generated for each encryption operation. This random value is combined with the user's password to make the derived key unique.

- PBKDF2 Iterations:
  The count parameter is set to 1,000,000, which means the key derivation function runs for 1,000,000 iterations. This significantly increases the computational effort required for brute-force attacks, enhancing the security of the derived key.

STEP 2: AES Encryption Setup

After deriving the key, the application initializes an AES cipher in CBC (Cipher Block Chaining) mode. CBC mode requires an Initialization Vector (IV), which is a random value that ensures that the same plaintext will encrypt to different ciphertexts every time, provided the IV is different. The `iv` is a 16-byte random value generated by the cipher object. It is critical for ensuring the uniqueness of ciphertexts and is stored alongside the encrypted data.

```
def encrypt_file(file_data, key, salt, file_extension):
    """Encrypt the file data in memory, including the original file extension."""
    cipher = AES.new(key, AES.MODE_CBC)
    iv = cipher.iv
```

Figure 5.6: Code snippet for AES setup.

STEP 3: File Data Preparation and Padding

To ensure the security and integrity of the encrypted file, the original file extension is prepended to the file data. This allows the application to retain the file's original format after decryption. The combined data (file extension + file content) must be padded to a multiple of the AES block size (16 bytes) to comply with AES encryption requirements.

```
try:
    # Store the extension at the beginning of the encrypted file
    extension_bytes = file_extension.encode('utf-8').ljust(16)
    ciphertext = cipher.encrypt(pad(extension_bytes + file_data, AES.bl
    encrypted_data = salt + iv + ciphertext
    return io.BytesIO(encrypted_data)
```

Figure 5.7: Code snippet for file padding and data preparation.

The `pad` function from `Crypto.Util.Padding` is used to pad the data to a multiple of 16 bytes. Padding is essential to fill any gaps in the last block of data, ensuring that the data fits perfectly into the AES encryption scheme.

STEP 4: Encrypting the File Data

The AES cipher encrypts the padded file data using the derived key and the generated IV. The resulting ciphertext is a binary format that is unreadable without the correct key and IV. The encrypted data is now stored in the 'ciphertext' variable after the result of applying the AES encryption algorithm to the padded plaintext. This ciphertext is the secure version of the original file content.

```
ciphertext = cipher.encrypt(pad(extension_bytes + file_data, AES.block_size))
```

Figure 5.8: Code snippet for encrypting the file data.

STEP 5: Concatenating Salt, IV, and Ciphertext:

The final encrypted data consists of the concatenation of the salt, IV, and ciphertext. This combination is necessary for decryption since both the salt and IV are needed to regenerate the key and decrypt the data.By concatenating the salt and IV with the ciphertext, the application ensures that all necessary components for decryption are bundled together in a single file.

```
encrypted_data = salt + iv + ciphertext
```

Figure 5.9: Code snippet for concatenating salt, IV and ciphertext.

STEP 6: Returning Encrypted Data to the User

The encrypted file is converted into an in-memory stream (`io.BytesIO`) and returned to the user for downloading.

```
return io.BytesIO(encrypted_data)
```

Figure 5.10: Code snippet to return encrypted data.

5.3.2 Decryption Process Implementation

The decryption process reverses the encryption steps to convert the encrypted file back into its original form, provided that the correct passkey is supplied. Below is the detailed explanation of the decryption implementation:

```
def decrypt_file(file_data, password):
    """Decrypt the file data in memory and extract the original extension."""
    try:
        salt = file_data[:16]
        iv = file_data[16:32]
        ciphertext = file_data[32:]

        key = get_key(password, salt)
        cipher = AES.new(key, AES.MODE_CBC, iv=iv)
        padded_plaintext = cipher.decrypt(ciphertext)
        plaintext = unpad(padded_plaintext, AES.block_size)

        # Extract the original extension
        original_extension = plaintext[:16].decode('utf-8').strip()
        file_content = plaintext[16:]

        return io.BytesIO(file_content), original_extension
    except Exception as e:
        return str(e), None
```

Figure 5.11: Code snippet for decryption function.

STEP 1: Extracting Salt, IV, and Ciphertext from Encrypted Data

When the user uploads an encrypted file for decryption, the application first extracts the salt, IV, and ciphertext from the file data. The salt and IV are required for regenerating the key and initializing the AES cipher.

```
salt = file_data[:16]
iv = file_data[16:32]
ciphertext = file_data[32:]
```

Figure 5.12: Code snippet for extraction of salt, IV, and ciphertext.

Salt and IV Extraction: The salt occupies the first 16 bytes, and the IV occupies the next 16 bytes of the encrypted file data. The remaining bytes represent the ciphertext.

STEP 2: Regenerating the Encryption Key

Using the extracted salt and the user-provided passkey, the application regenerates the original encryption key using PBKDF2. This key must match the one used during encryption to successfully decrypt the file.If the user provides the correct passkey, the regenerated key will match the original key, allowing the decryption process to proceed.

```
key = get_key(password, salt)
```

Figure 5.13: Code snippet for regeneration of encryption key.

STEP 3: Initializing the AES Cipher for Decryption

The application initializes an AES cipher in CBC mode using the regenerated key and the extracted IV.The IV must match the one used during encryption. It ensures that the decryption process reverses the encryption correctly, block by block.

```
cipher = AES.new(key, AES.MODE_CBC, iv=iv)
```

Figure 5.14: Code snippet for AES initialization.

STEP 4: Decrypting the Ciphertext

The ciphertext is decrypted using the AES cipher, producing the padded plaintext (original file extension + original file content). This step reverses the encryption process, converting the ciphertext back into its original, padded format. If the key or IV is incorrect, the output will be unintelligible.

```
padded_plaintext = cipher.decrypt(ciphertext)
```

Figure 5.15: Code snippet for decryption process.

STEP 5: Removing Padding and Extracting the Original File Extension

After decrypting the data, the padding is removed to recover the original file content. The first 16 bytes of the decrypted data represent the original file extension, which is extracted and used to reconstruct the file.

```
plaintext = unpad(padded_plaintext, AES.block_size)

# Extract the original extension
original_extension = plaintext[:16].decode('utf-8').strip()
file_content = plaintext[16:]
```

Figure 5.16: Code snippet getting back the original file.

The `unpad` function removes any padding added during encryption. This step is critical for ensuring that the decrypted data matches the original content exactly.

STEP 6: Returning Decrypted File to the User

The decrypted file is then prepared for download with its original extension restored, ensuring that the file remains usable in its original format after decryption.

```
return io.BytesIO(file_content), original_extension
```

Figure 5.10: Code snippet to return decrypted data.

5.3.3 Security Measures in Encryption and Decryption

- Zero-Knowledge Principle:
  The system ensures that no sensitive information (such as user passwords) is stored on the server. Instead, it uses the hashed passkey combined with a salt for encryption and decryption.

- PBKDF2 Key Strengthening:
  The use of PBKDF2 with a high iteration count adds computational complexity, making it resistant to dictionary and brute-force attacks.

- AES-256 Encryption:
  AES-256 is a highly secure encryption standard, resistant to most known cryptographic attacks.

5.4 Server and Application Implementation

5.4.1 Flask Web Application

The application is hosted on a local Flask development server configured with the following settings:

- Server Configuration

  The server is run in debug mode for development purposes

```
if __name__ == '__main__':
    app.run(debug=True)
```

Figure 5.17: Code snippet to run flask web application.

  This setting allows for real-time error tracking and automatic reloading of code changes.

- Static File Management

  Static files, such as CSS, JavaScript, and video backgrounds, are served using Flask's `url_for()` function to ensure proper referencing.

5.4.2 Application logic flow

Below code snippet explains the core logic for handling file encryption and decryption requests in a Flask web application.

```python
@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        if 'file' not in request.files:
            return redirect(request.url)
        file = request.files['file']
        password = request.form.get('password')
        service = request.form.get('service')

        if file and password:
            file_data = file.read()
            original_filename = file.filename
            original_extension = os.path.splitext(original_filename)[1]  # Get the file extension
            salt = get_random_bytes(16)
            if service == 'Encrypt':
                key = get_key(password, salt)
                encrypted_file = encrypt_file(file_data, key, salt, original_extension)
                encrypted_file.seek(0)  # Reset stream position to the beginning
                encrypted_filename = f"{os.path.splitext(original_filename)[0]}.enc"
                flash(f'File encrypted successfully! Click "Download File" to get your file.')
                return send_file(encrypted_file, as_attachment=True, download_name=encrypted_filename)
            elif service == 'Decrypt':
                decrypted_file, original_extension = decrypt_file(file_data, password)
                if original_extension is None:
                    # Show a JavaScript dialog box on decryption failure
                    error_message = "Decryption failure - Wrong passkey or invalid file type"
                    return render_template('error.html', error_message=error_message)
                decrypted_file.seek(0)  # Reset stream position to the beginning
                decrypted_filename = f"{os.path.splitext(original_filename)[0]}_decrypted{original_extension}"
                flash(f'File decrypted successfully! Click "Download File" to get your file.')
                return send_file(decrypted_file, as_attachment=True, download_name=decrypted_filename)
            else:
                abort(400, "Invalid service option")
        else:
            abort(400, "File or password missing")

    return render_template('index.html')
```

Figure 5.18: Code snippet to handle request from web application.

Figure 5.18 shows the section of code that is designed to handle file encryption and decryption tasks within a Flask web application. It processes user requests based on the HTTP method and service type selected (either encryption or decryption), ensuring that files are appropriately handled and secure. When the user submits a form (via POST request), the code first checks if a file is included in the request. If no file is present, the user is redirected to the same page. The file, password, and selected service are then retrieved from the request. If either the file or password is missing, or if the service option is invalid, the application responds with a 400 Bad Request error.

For a valid request, the file's content is read and its original filename and extension are extracted. A 16-byte salt is generated, which is crucial for the encryption process. Depending on the selected service, the code either encrypts or decrypts the file. If the encryption service is selected, the password and salt are used to derive an encryption key through the get_key function. The file is then encrypted using this key and the original file extension. After encryption, the encrypted file is made available for download, and a success message is displayed to the user.

In the case of decryption, the code attempts to decrypt the file using the provided password. If decryption fails—possibly due to an incorrect password or an invalid file type—the user is directed to an error page with an appropriate message. If successful, the decrypted file is prepared for download with a new filename indicating it has been decrypted, and a success message is shown. The application ensures that file streams are correctly reset to the beginning before sending them back to the user. This approach allows the user to download the processed file seamlessly.

Finally, the index.html template is rendered for GET requests, providing the main interface for file operations. The Flask application is configured to run in debug mode, which aids in development by providing detailed error messages and automatic reloading.

Overall, this code effectively handles file encryption and decryption tasks, managing both user interactions and backend processing securely and efficiently.

5.5 Testing Strategies

A few testing strategies have been implemented to ensure the quality and functionally of this developed file encryption tool. The tests involved are unit testing, integration testing, system testing and user acceptance testing (UAT).

5.5.1 Unit Testing

Unit Testing focuses on verifying the smallest testable parts of the application, such as individual functions or methods, in isolation. For the "StevHash" application, unit testing ensures that core functionalities like key generation, encryption, decryption, and error handling methods work correctly on their own without dependencies. Unit tests are usually automated and written by developers to catch errors early in the development process.

Test Description	**Test Actions/Input**	**Expected Output**	**Actual Output**	**Pass/Fail**
Test Key Generation Function (`get_key`)	Input password and salt. Password is user-input.	Returns a 32-byte key for AES256 encryption	Returns a 32-byte key for AES256 encryption	Pass
Test Encryption Function	Input file data, key, salt, and file	Returns encrypted file data in	Returns encrypted file	Pass

(`encrypt_file`)	extension	`BytesIO` format	data in `BytesIO` format	
Test Decryption Function (`decrypt_file`)	Input encrypted file data and password	Returns original file data and file extension	Returns original file data and file extension	Pass
Test Error Handling for Decryption Failures	Input incorrect password or invalid file data	Returns error message indicating decryption failure	Returns error message indicating decryption failure	Pass

Table 5.1: Result of Unit Testing approach..

5.5.2 Integration Testing

Integration Testing is performed to ensure that different modules or components of the application work together as expected. In the context of "StevHash," integration testing verifies the interaction between the Flask backend, encryption/decryption services, and the user interface. It checks that data flows between these components seamlessly and that combined functionalities do not produce unintended behaviors.

Test Description	Test Actions/Input	Expected Output	Actual Output	Pass/Fail
Test Flask Backend and Frontend Interaction	Submit file and password via the form on `index.html`	Backend processes the request, encrypts/decrypts file, sends response	Backend processes the request, encrypts/decrypts file, sends response	Pass
Test Data Flow Between Form and Backend	Select 'Encrypt' option, upload a file, input password	Encrypted file available for download	Encrypted file available for download	Pass
Test Handling of Encrypted File Downloads	Download encrypted file from the application	Encrypted file downloads correctly	Encrypted file downloads correctly	Pass
Test Handling of Decrypted File Downloads	Download decrypted file from the application	Decrypted file downloads correctly	Decrypted file downloads correctly	Pass

Table 5.2: Result of Integration Testing approach..

5.5.3 System Testing

System Testing involves testing the complete application as a whole to ensure that it meets the specified requirements. For "StevHash," system testing focuses on the full web application, including all integrated modules and external dependencies (like video backgrounds and CSS). The goal is to validate that the entire application works as intended when deployed in the environment where it will be used.

Test Description	Test Actions/Input	Expected Output	Actual Output	Pass/Fail
Test Overall Application Functionality	Perform encryption and decryption on different file types	All files are encrypted and decrypted successfully	All files are encrypted and decrypted successfully	Pass
Test Browser Compatibility	Access the application on Chrome, Firefox, Edge, Safari	Application works correctly on all tested browsers	Application works correctly on all tested browsers	Pass
Test Responsive Design and UI Behavior	Resize browser window, test on different devices	UI adjusts correctly and remains usable on all devices	UI adjusts correctly and remains usable on all devices	Pass

Table 5.3: Result of System Testing approach..

5.5.4 User Acceptance Testing(UAT)

User Acceptance Testing (UAT) is the final phase of testing, where the application is tested from the end user's perspective to ensure it meets their expectations and requirements. In "StevHash," UAT would involve real users (or stakeholders) performing encryption and decryption tasks to confirm that the user interface is intuitive and that the application functions correctly according to user needs.

Test Description	Test Actions/Input	Expected Output	Actual Output	Status
Tester 1: Alice				
Test Encryption of a Text File	Upload a .txt file, input password, execute 'Encrypt'	Encrypted file downloaded successfully.	Encrypted file downloaded successfully.	Pass

Test Decryption of a Text File	Upload the encrypted .txt file, input correct password and execute the decryption.	Decrypted file is correctly restored	Decrypted file is correctly restored	Pass
Test Error Handling for Incorrect Password	Upload encrypted file, input incorrect password and execute decryption.	Error message displayed indicating decryption failure	Error message displayed indicating decryption failure	Pass
Tester 2: Bob				
Test Upload and Encryption of an Image File	Upload a .jpg file, input password, select 'Encrypt'	Encrypted image file available for download	Encrypted image file available for download	Pass
Test Upload and Decryption of an Image File	Upload the encrypted .jpg file, input correct password	Decrypted image file is correctly restored	Decrypted image file is correctly restored	Pass
Test File Size Limits	Upload a large file (100MB) for encryption	File is encrypted and available for download	File is encrypted and available for download	Pass
Tester 3: Charlie				
Test UI and UX for Encryption/Decryption	Navigate the application, perform encryption and decryption	Tester is able to perform functions in short time and feel relax/easy	Tester is able to perform functions in short time and feel relax/easy	Pass
Test Responsiveness Across Devices	Access the application on mobile and tablet devices	UI adapts correctly on all devices	UI adapts correctly on all devices	Pass
Test Error Handling for Missing Inputs	Submit the form with missing file or password	Appropriate error messages are displayed	Appropriate error messages are displayed	Pass
Tester 4: Dana				
Test Compatibility with Different Browsers	Access the application using Chrome, Firefox, Edge, Safari	Application functions correctly on all browsers	Application functions correctly on all browsers	Pass

Test Performance and Speed	Perform multiple encryptions and decryptions in a row	Application remains responsive and quick	Application remains responsive and quick	Pass

5.6 Chapter Summary and Evaluation

In this chapter, we outlined the comprehensive process of implementing and testing the StevHash web application, designed for secure file encryption and decryption using AES-256 encryption. We began with an overview of the system's architecture, detailing the use of Python with the Flask framework for backend development and HTML, CSS, and JavaScript for frontend design. The implementation section provided an in-depth explanation of the encryption and decryption processes, including key derivation using PBKDF2, AES cipher initialization, data preparation, and security measures that adhere to the zero-knowledge principle.

The chapter also covered the handling of user interactions, error management, and feedback mechanisms to ensure a smooth user experience. The testing strategies discussed, including unit testing, integration testing, system testing, and user acceptance testing, demonstrated the thorough validation of the system's functionality, performance, and user interface across different scenarios and devices. These tests confirmed the application's ability to securely encrypt and decrypt files, handle errors appropriately, and provide a user-friendly experience.

Overall, this chapter highlights the meticulous approach taken in developing and refining the StevHash application, ensuring it meets the highest standards of security, usability, and functionality. The successful completion of various testing phases validates the system's readiness for real-world deployment and its potential to provide users with a reliable and secure file encryption and decryption service.

Chapter 6 *(Not applicable)*

System Deployment

6 System Deployment

In this chapter it is not applicable for my project as my web application is not a real-life industrial project. Therefore, my project of the web application is developing for the testing of online file encryption/decryption service.

Chapter 7

Discussions and Conclusion

7 Discussions and Conclusion

7.1 Summary

This project aimed to develop a secure web-based encryption tool named StevHash that utilizes the AES-256 encryption standard to prevent unauthorized access to sensitive data. The solution incorporates the zero-knowledge principle to ensure that user passkeys are not stored, enhancing privacy and security. The project was implemented using Python with Flask for the backend and HTML, CSS, and JavaScript for the frontend. The Agile methodology was employed to allow iterative development and refinement, ensuring flexibility and responsiveness to emerging requirements and challenges. Overall, the project successfully delivered a robust and user-friendly encryption tool that meets the defined objectives.t.

7.2 Achievements

The project achieved its primary objectives by developing a functional web application that provides secure file encryption and decryption using AES-256. It effectively applied the zero-knowledge principle by ensuring user passwords are never stored on the server, only the generated hash values are temporarily used. The tool was tested rigorously through unit, integration, system, and user acceptance testing to validate its performance, security, and usability. Strengths of the system include its strong encryption method, ease of use, and secure handling of user data. A noted weakness is the lack of support for large files beyond a certain size, which could limit its applicability in some contexts.

7.3 Contributions

StevHash brings innovation by combining strong AES-256 encryption with zero-knowledge protocols in a web-based environment, offering a user-friendly interface that simplifies secure file handling for users with varying technical skills. This system is necessary as it provides a secure way to encrypt and decrypt files without exposing user data to potential breaches, making it highly relevant in the age of increasing cyber threats. Its marketability lies in its applicability for individuals and organizations looking for a secure, easy-to-use encryption tool that does not compromise user privacy.

7.4 Limitations and Future Improvements

While StevHash effectively addresses the need for secure file encryption, there are several limitations that need to be addressed in future developments:

1. Insufficient Testing:

   Although the web application has undergone basic functional testing, it lacks a comprehensive Vulnerability Assessment and Penetration Testing (VAPT) assessment. Conducting VAPT is crucial to identifying potential security weaknesses and ensuring the safe operation of the application in various threat environments. Future improvements should include a thorough VAPT to strengthen the security posture of StevHash.

2. User Interface Limitations

   Currently, the user interface (UI) of the StevHash web application is optimized only for desktop and laptop devices. It does not support smartphone interfaces effectively, leading to usability challenges on smaller screens. Future development should focus on making the UI responsive and adaptive to different screen sizes and devices, enhancing accessibility and user experience across all platforms.

3. Limited File Size Support

   The current implementation of StevHash supports only small to medium-sized files, approximately up to 2GB, which restricts its use for encrypting large datasets. To overcome this limitation, future improvements could focus on optimizing the encryption process to handle larger files efficiently without compromising performance or security.

4. Exploring Advanced Encryption Algorithms

   Encryption technologies continue to evolve, and future iterations of StevHash could benefit from integrating more advanced encryption algorithms such as XChaCha20. XChaCha20 offers several advantages over AES-256, including better performance on some platforms, resistance to timing attacks, and more robust handling of nonce misuse.

Implementing XChaCha20 could enhance both the security and performance of the application.

5. Enhanced Key Management

   While the current version of StevHash employs a basic key management mechanism using PBKDF2 for key derivation, there is a need for more secure and scalable key management solutions. Future improvements could involve implementing hardware security modules (HSMs) or integrating with external key management services (KMS) to provide enhanced security, better key rotation policies, and improved overall key management.

These future improvements would not only enhance the functionality and security of StevHash but also expand its applicability to a broader range of use cases, ensuring it remains a robust and reliable encryption tool for modern data security challenges.

7.5 Issues and Solutions

Several challenges were encountered during the project, including technical issues with implementing secure key management and ensuring the zero-knowledge principle was maintained. These were addressed by employing well-documented cryptographic libraries and following best practices in secure coding. The user interface required refinement to ensure accessibility and usability across different devices, which was solved through iterative testing and user feedback. Lessons learned include the importance of agile practices for accommodating evolving requirements and the need for continuous security assessment to address potential vulnerabilities.

This chapter summarizes the project outcomes and highlights the achievements, contributions, limitations, and lessons learned. It provides a foundation for future development and potential research in enhancing secure file encryption solutions.

References

I. Bless, G. (2023, March 11). *Challenges and limitations of encryption*. Medium. https://gupta-bless.medium.com/challenges-and-limitations-of-encryption-33b79ed046d4

II. DataGrail. (2022, May 4). *6 Common Data Privacy Issues*. DataGrail. https://www.datagrail.io/blog/data-privacy/data-privacy-issues/

III. Ernstberger, J., Chaliasos, S., Zhou, L., Jovanovic, P., & Gervais, A. (2024). *Do You Need a Zero Knowledge Proof?* https://eprint.iacr.org/2024/050.pdf

IV. Forta. (2022, January 24). Should You Be Using File Encryption Software? *Should You Be Using File Encryption Software?* https://www.globalscape.com/blog/should-you-be-using-file-encryption-software

V. Ikhwan, A., Aliana A. Raof, R., Ehkan, P., Yacob, Y., & Aslami, N. (2023). Implementation of image file security using the advanced encryption standard method. *Indonesian Journal of Electrical Engineering and Computer Science*, *31*(1), 562. https://doi.org/10.11591/ijeecs.v31.i1.pp562-569

VI. Jena, B. K. (2023, February 9). *What Is AES Encryption and How Does It Work? - Simplilearn*. Simplilearn.com. https://www.simplilearn.com/tutorials/cryptography-tutorial/aes-encryption

VII. Kaspersky. (2020). *What is Data Encryption?* Kaspersky.com. https://www.kaspersky.com/resource-center/definitions/encryption

VIII. Louis, D. (2020, September 10). *Should I Encrypt Sensitive Files on My Computer?* Www.experian.com. https://www.experian.com/blogs/ask-experian/should-i-encrypt-sensitive-files-on-my-computer/

IX. Poller, J. (2023, February 2). *Understanding the importance of data encryption | TechTarget*. Security. https://www.techtarget.com/searchsecurity/opinion/Understanding-the-importance-of-data-encryption

X. Rebecca , H. (2020, March 21). *Top 4 Reasons Encryption Is Not Used*. Privacysecuritybrainiacs.com.

https://privacysecuritybrainiacs.com/privacy-professor-blog/top-four-reasons-encryption-is-not-used/

XI. Shavell, R. (2022, August 30). *Council Post: The Daily Reality Of Modern Privacy Problems*. Forbes. https://www.forbes.com/sites/forbestechcouncil/2022/08/30/the-daily-reality-of-modern-privacy-problems/?sh=284702793dfb

XII. Smirnoff, P., & Turner, D. M. (2020, January 3). *Symmetric Key Encryption - why, where and how it's used in banking*. Cryptomathic. https://www.cryptomathic.com/news-events/blog/symmetric-key-encryption-why-where-and-how-its-used-in-banking

XIII. Soewito, B., & Marcellinus, Y. (2020). IoT security system with modified Zero Knowledge Proof algorithm for authentication. *Egyptian Informatics Journal*. https://doi.org/10.1016/j.eij.2020.10.001

XIV. Sunila, Dr. Shubhamangala. (2023, September 30). *Cryptography Challenges*. Www.linkedin.com. https://www.linkedin.com/pulse/cryptography-challenges-dr-shubhamangala-sunila

XV. Ware, R. (2022, July 11). *What Is Zero-Knowledge Encryption, and Why Should I Choose It?* How-to Geek. https://www.howtogeek.com/811527/what-is-zero-knowledge-encryption/

XVI. Wickramasinghe, S. (2023, May 15). *RSA Algorithm in Cryptography: Rivest Shamir Adleman Explained*. Splunk-Blogs. https://www.splunk.com/en_us/blog/learn/rsa-algorithm-cryptography.html

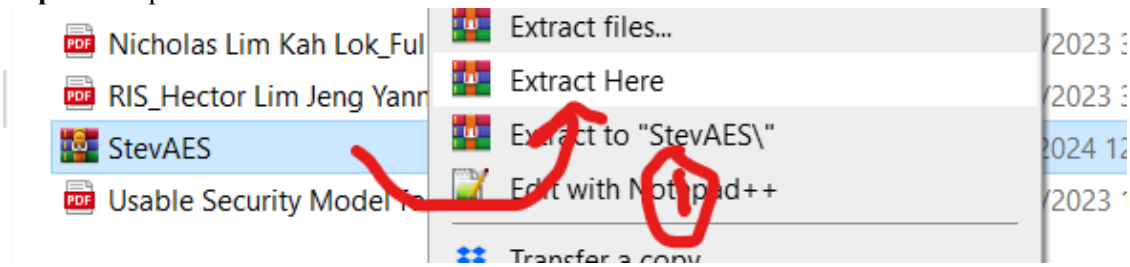
XVII. Yehya, D., & Joudi, M. (2020). *AES Encryption: Study & Evaluation*. https://www.researchgate.net/profile/Mohamad-Joudi-2/publication/346446212_AES_Encryption_Study_Evaluation/links/5fd14d81299bf188d406c8d3/AES-Encryption-Study-Evaluation.pdf

XVIII. Zodpe, H., & Shaikh, A. (2021). *A Survey on Various Cryptanalytic Attacks on the AES Algorithm*. International Journal of Next-Generation Computing - Special Issue, 12(2), 115-123. https://doi.org/10.47164/ijngc.v12i2.202

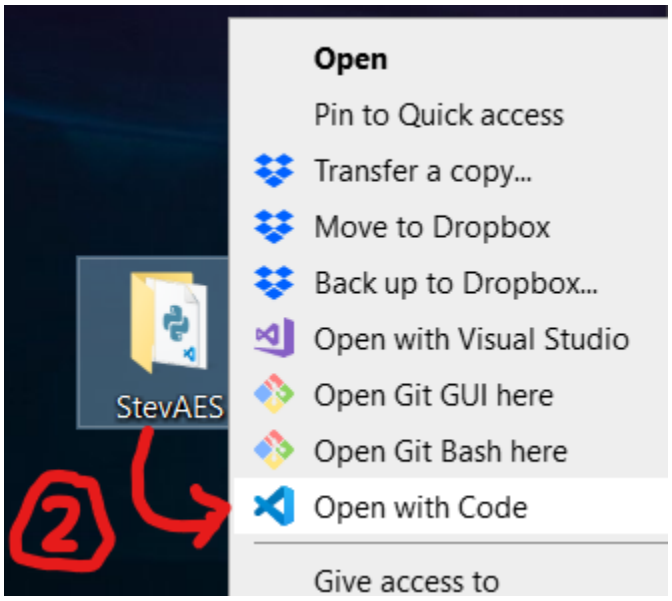
Appendices

1. How to run the Web Application

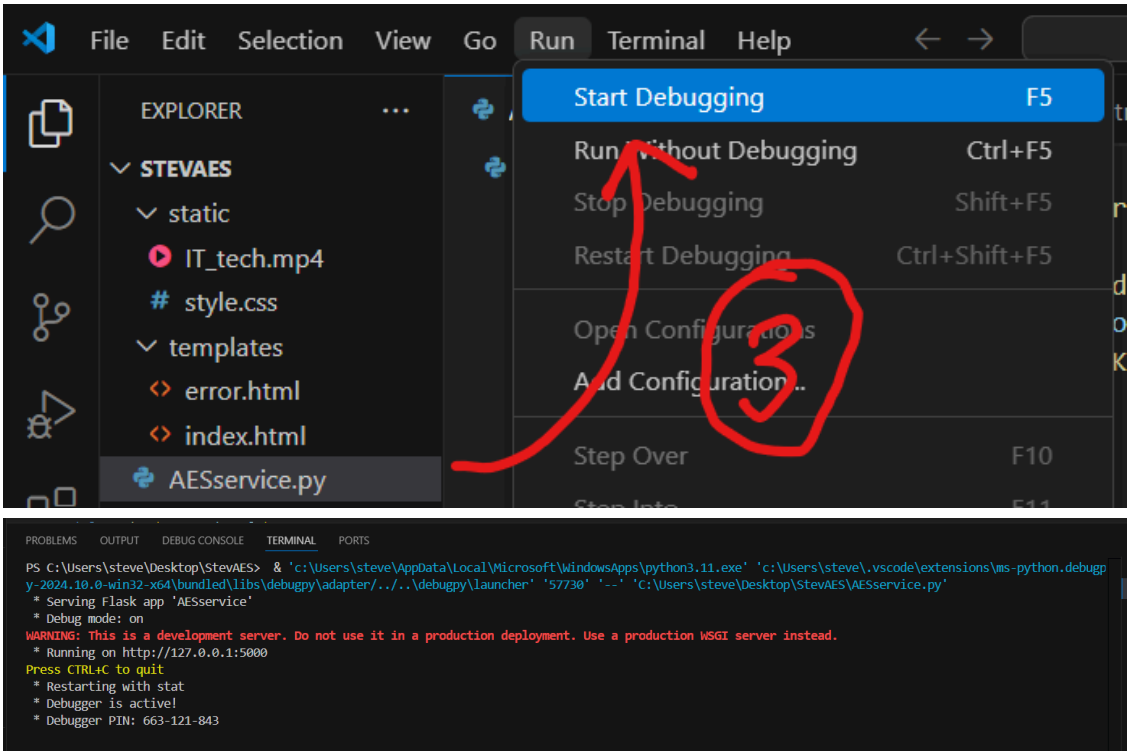
Step 1: Unzip/Extract the file



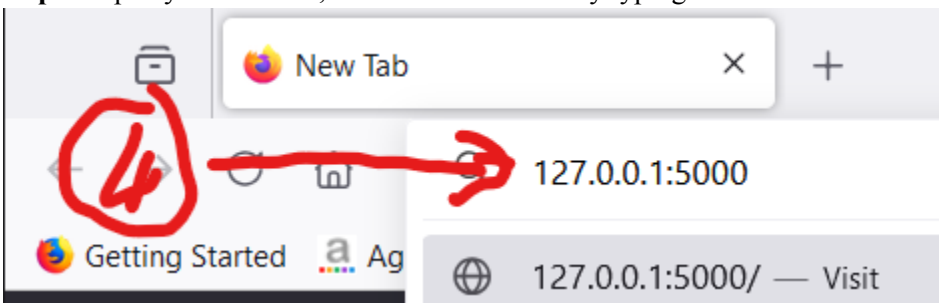
Step 2: Open the file with Visual Studio Code



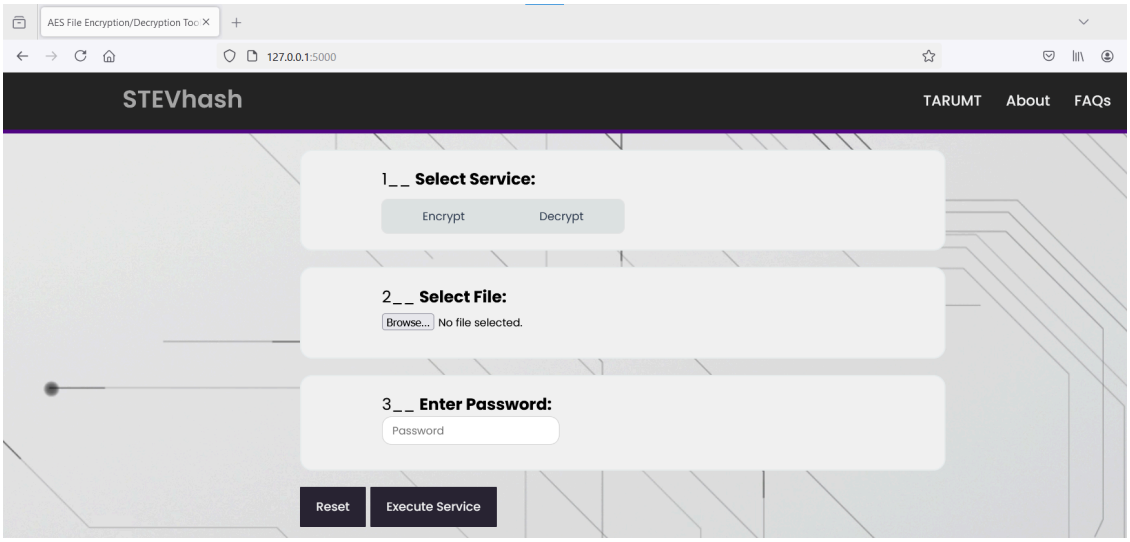
Step 3: Go to main code(AESservice.py), click Run (F5)



Step 4: Open your browser, connect to localhost by typing 127.0.0.1:5000



You can test the web application now.


AES File Encryption/Decryption Too
127.0.0.1:5000
STEVhash
TARUMT
About
FAQs
1__ Select Service:
Encrypt
Decrypt
2__ Select File:
Browse...
No file selected.
3__ Enter Password:
Password
Reset
Execute Service

This page is intentionally left blank to indicate the back cover. Ensure that the back cover is black in color.